
**Department of Computer Science**
**Software Engineering**

Phase 1

Software Requirement Specification Document

# InnovaCode

Name and ID:

Alia Khamis U19102199

Amna Rashed Almazrouei U23103547

Asma Al-Ali U22100601

Hasna Imad U23100997

Hind Almarzooqi U21201139

Namah Sultan U22100470

Reyam Sahib U23105903

# Table of Contents

## 1.0 Introduction

The purpose of this document is to describe all the requirements of the Shared Farm Web Application. This report establishes the data, functional, and behavioral specifications of the system.

### 1.1 Goals and objectives

The project's primary goal is to provide a platform that encourages sustainability, by connecting farm owners, customers, and investors. Objectives include:

- Allowing shareholders to order free produce and manage their farm shares.

- Offering and online store for affordable organic local products.

- Providing investment opportunities.

- Using dashboards, voting systems, and reports to provide transparency.

- Integrating AI features for forecasts, tips, and recommendations.

- Encourage community awareness and sustainability through workshops.

The Shared Farm Web Application aims to establish a digital platform that allows a shared ownership of an organic farm in the UAE by combining farm management, e-commerce, and AI analytics. Providing fresh products at a reasonable price, transparent farm management, and sustainable farming methods.

### 1.2 Statement of scope

The purpose of the Shared Farm Web Application is to offer a platform for transparent organic food management and collective farm ownership. Product details, farm share information, user registration data, and investment contributions are important inputs. Confirmed deliveries, updated dashboards, and transparency records are among the outputs of the system's processing of this data, which is used to maintain ownership, distribute free produce, process customer orders, track investments, and create reports. User identification, shareholder dashboards, e-commerce capabilities, investment features, and AI demand predictions are all crucial from a development standpoint. While future criteria like IoT connection and blockchain-based investment tracking are outside the current semester's purview, desirable requirements include the voting system, sustainability effect tracker, and AI-driven recipe and storage suggestions.

Major Processing Functions:

| Function | Priority | Function Description |
|---|---|---|
| Create an account and Login | Essential | Users can create accounts or log in to an existing account and access their dashboard. |
| Product Catalogue and Browsing | Essential | Customers & shareholders can browse the organic products filtered by category, price, or availability. |
| Placing Orders | Essential | The ability for a user to add items to their cart, complete a secure payment, and schedule delivery, confirming the order is placed. |
| Box Subscription | Essential | Customers can choose a subscription plan, complete payment, and receive scheduled deliveries. |
| Shareholder Produce | Essential | Shareholders can request their entitled produce, which is scheduled for delivery with no additional costs. |
| Exchange Products | Essential | Shareholders can list surplus items from their dashboard for others to trade or purchase. |
| Manage Investments | Essential | Investors fund selected projects, contribute funds, and track progress/funding status. |
| Dashboard and Reports | Essential | Users can view dashboards summarizing production, deliveries, investments, and green impact. |
| AI Features | Essential | Users receive smart recommendations for recipes & storage tips, as well as demand forecasts and prediction. |
| Educational Events and Insights | Desirable | A learning resource for users to read blogs, register for workshops, and join farm events. |
| Impact Tracker | Desirable | Users can view measured sustainability contributions, such as CO2 savings and waste reduction. |
| Community awareness | Desirable | The ability for users to learn about Shared Farm’s mission & values. |
| Contact Form | | Customers can send feedback or inquiries. |

### 1.3 Software context

The Shared Farm Web Application falls under the categories of community involvement, farm management, and e-commerce. By providing a digital platform for community farm ownership, product distribution, and investment, it aims to bring together technology and agriculture. From a strategic standpoint, the system tackles a number of challenges relevant to the United Arab Emirates, including the necessity for sustainable farming practices, the significant reliance on imported organic produce, and urban residents' restricted access to farmland. In addition to increasing productivity, the application supports national objectives for food security, sustainability, and digital transformation by integrating AI for demand forecasting and transparency tools for reporting.

### 1.4 Major constraints

When using a digital platform to track investments and produce, potential users might be reluctant to approve of a shared ownership model for farming. The system must manage growing numbers of shareholders, clients, and transactions without compromising performance, which makes ensuring scalability another difficulty. Another constraint is integration with external services such as payment providers and delivery systems, which may raise compatibility issues. Legal issues are also present because, in order to function properly, the platform needs to go by UAE laws regarding investment models, food safety, and e-commerce.

## 2.0 Usage scenario

The Shared Farm Web Application will accommodate many user types who engage with the system in various ways. The next subsections elaborate on these use cases, which serve as the foundation for the system's functional needs.

### 2.1 User profiles

The Shared Farm Web Application will be used by five main categories of users.

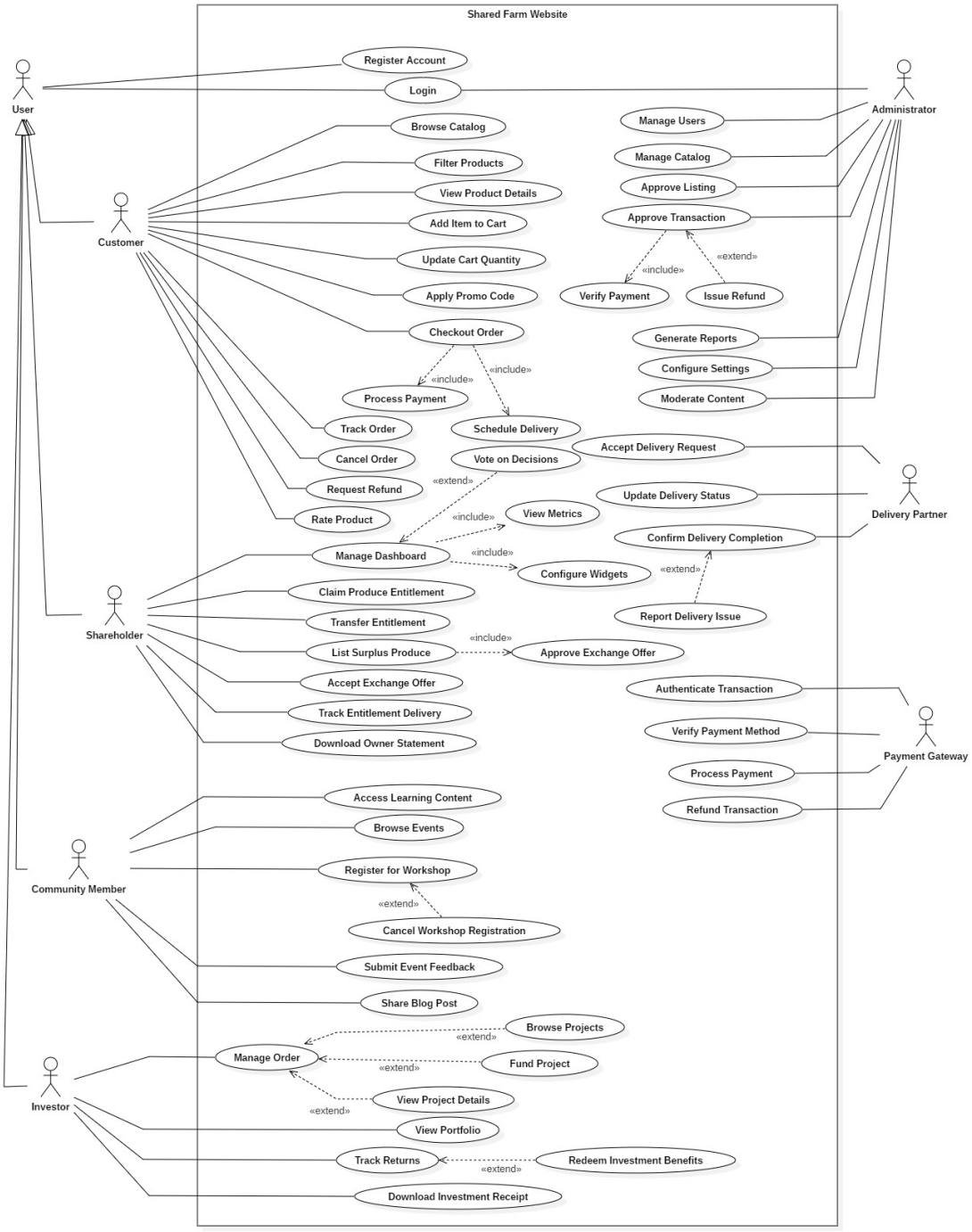
1. Shareholders (Owners): owners who buy shares in the farm can view their dashboards, receive free produce from their shares, and take part in decision making.
2. Customers: Regular users who can browse the online store, buy products, and sign up for seasonal produce boxes with delivery or pickup choices but do not own shares.
3. Investors: Customers who make financial contributions for sustainability initiatives or equipment. Depending on their investment, they may receive profit shares, free produce, or discounted goods in exchange.
4. Administrators are system managers in charge of managing every aspect of the business. They create reports, validate information, authorize transactions, and guarantee platform transparency.

5. Community Members: Users who engage with the system's instructional elements. They have access to sustainability information, blogs, and tutorials. They can also sign up for workshops and farm events.

### 2.2 Use-cases

All use-cases for the software are presented.

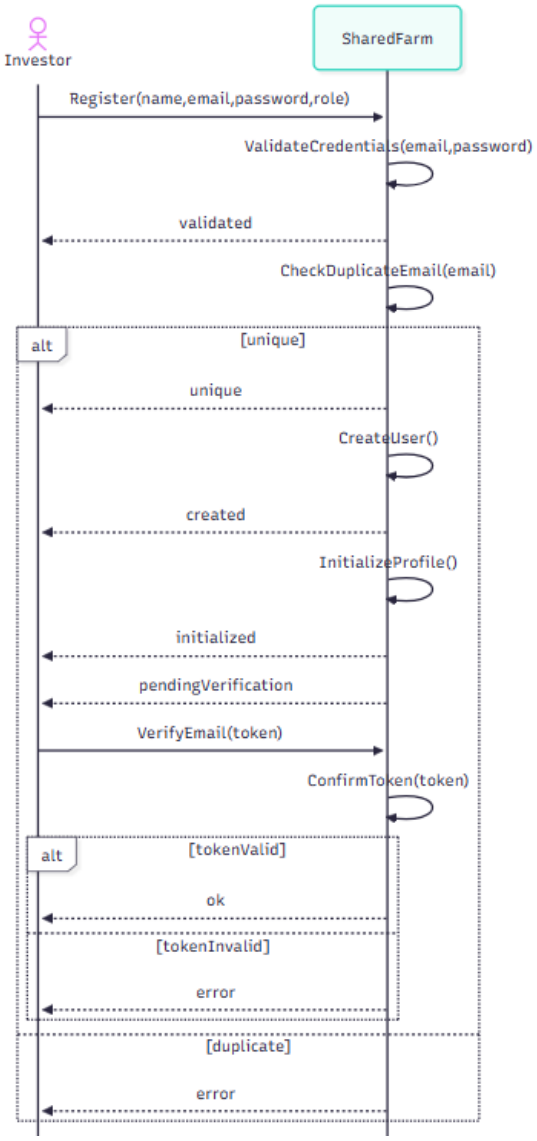
# Use case diagram:



# Use case Descriptions:

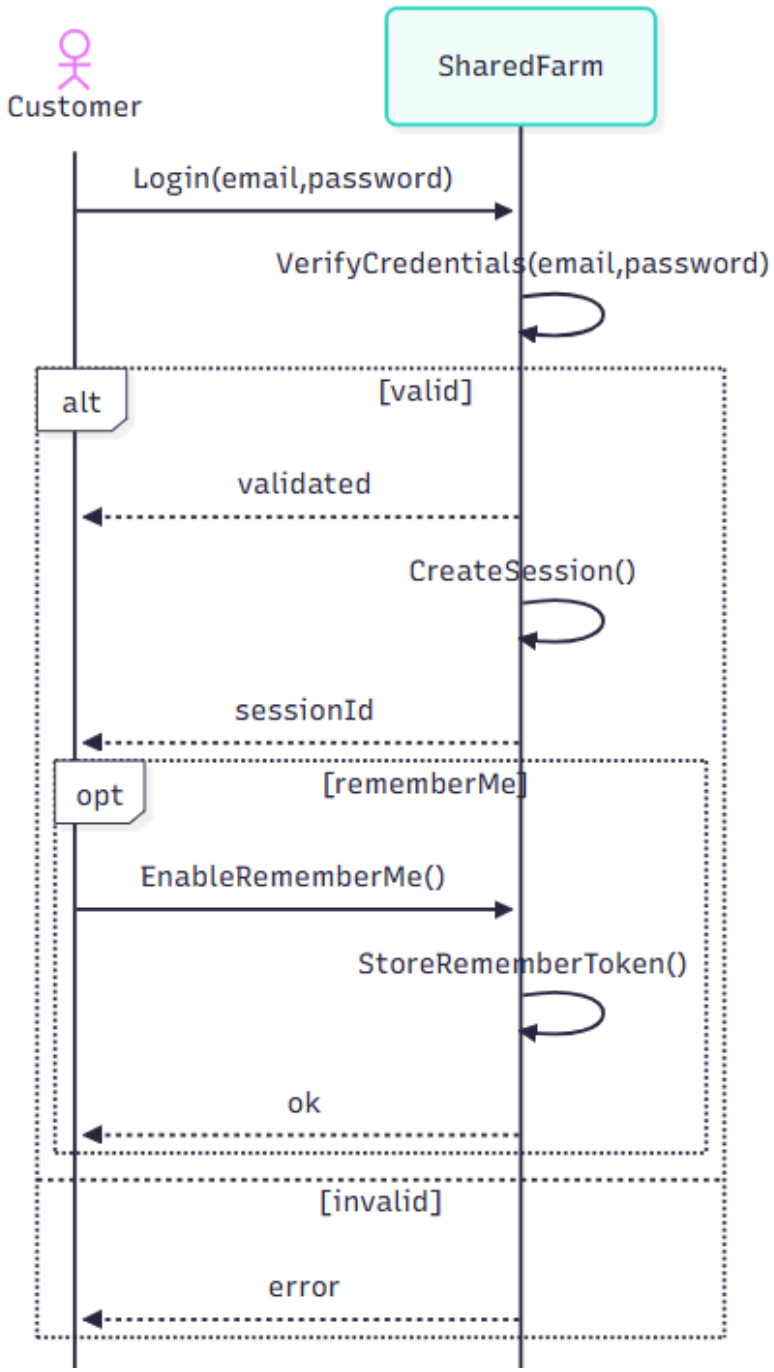
## 1. Register Account (Investor):

The actor (Investor; same flow for Customer, Shareholder, Administrator, and Community Member) submits name, email, password, and role. The system validates credentials, checks for duplicate email, creates the user, initializes the profile, and requests email verification. If the token is valid the account is activated and returns ok; if the email is taken or the token is invalid, the system returns error.



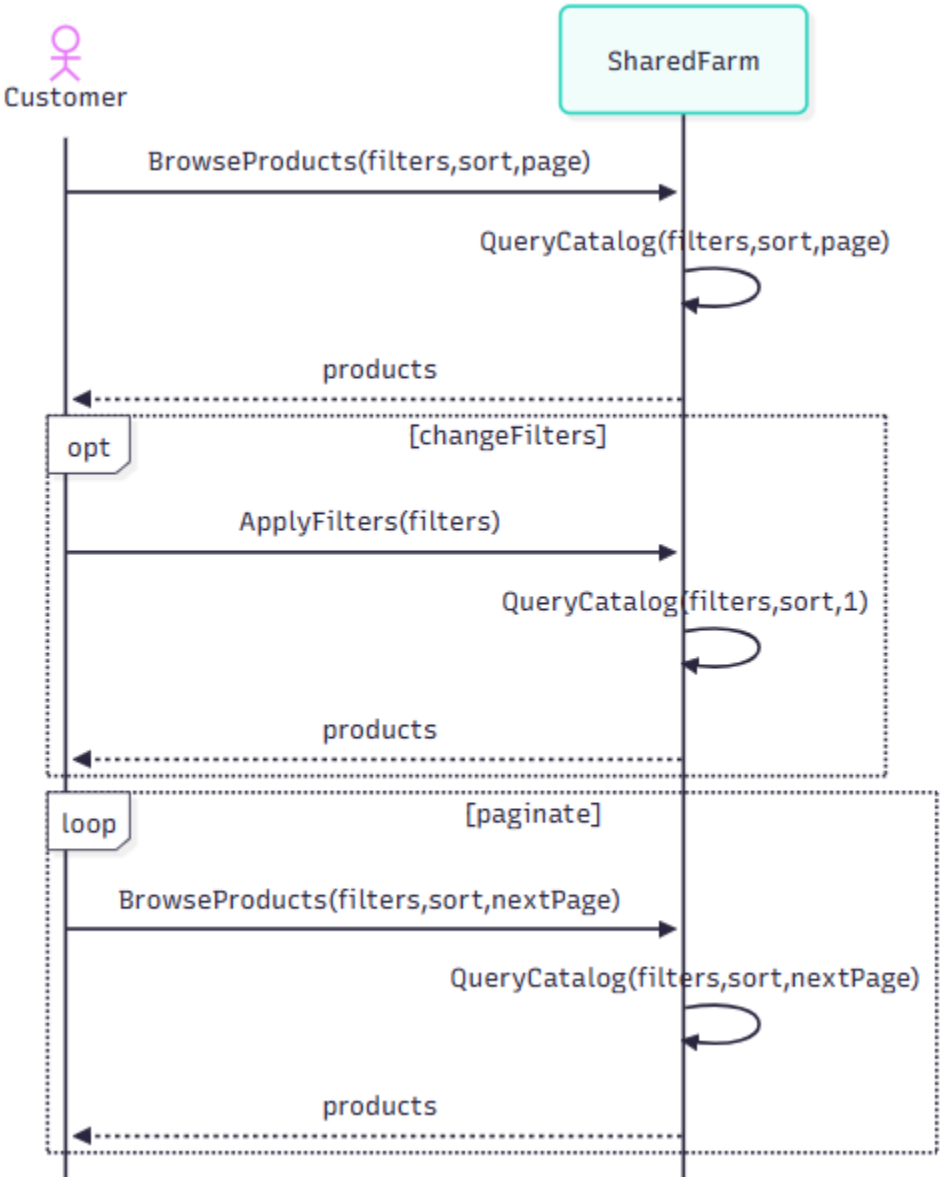
2. **Login (Customer)**:

The actor (Customer; same for all registered roles) enters email and password. The system verifies credentials and, if valid, creates a session and optionally stores a remember-me token, returning a session ID and ok. If the credentials are invalid, the system returns error.



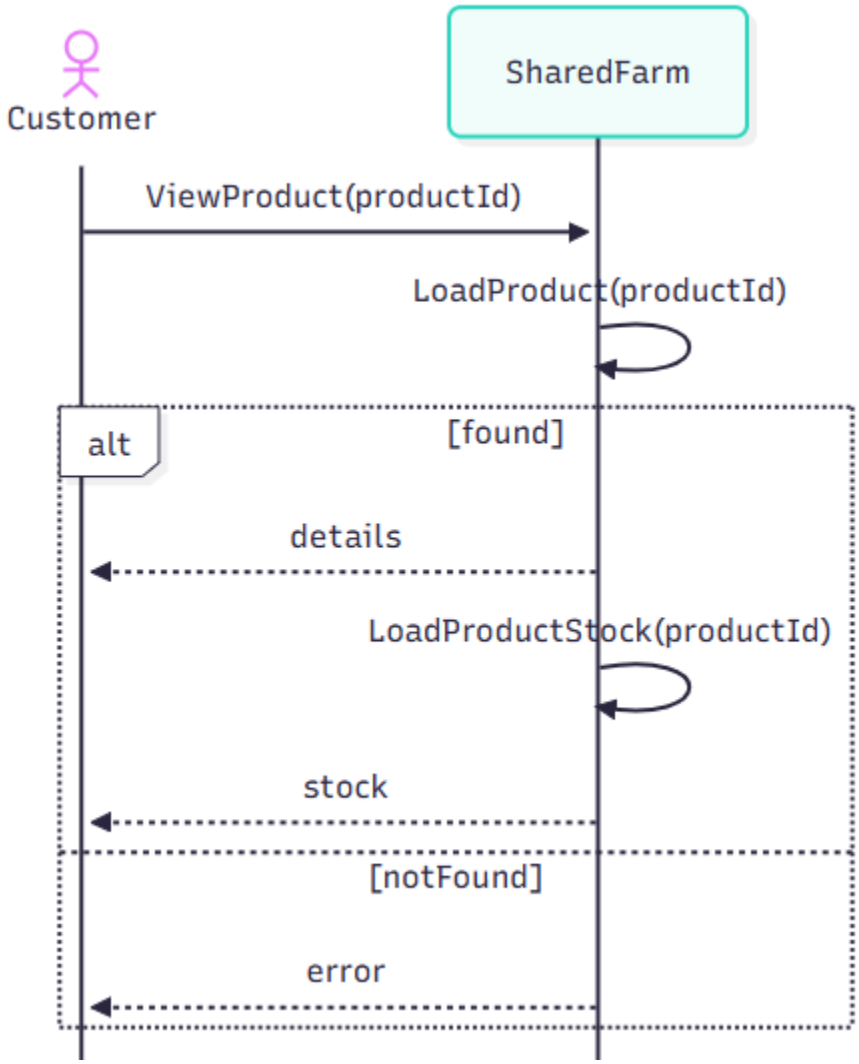
3. **Browse Catalog (Customer)**:

The actor requests the product list with filters, sort, and page. The system queries the catalog and returns products. The actor can refine filters or paginate; each request re-queries and returns updated products.



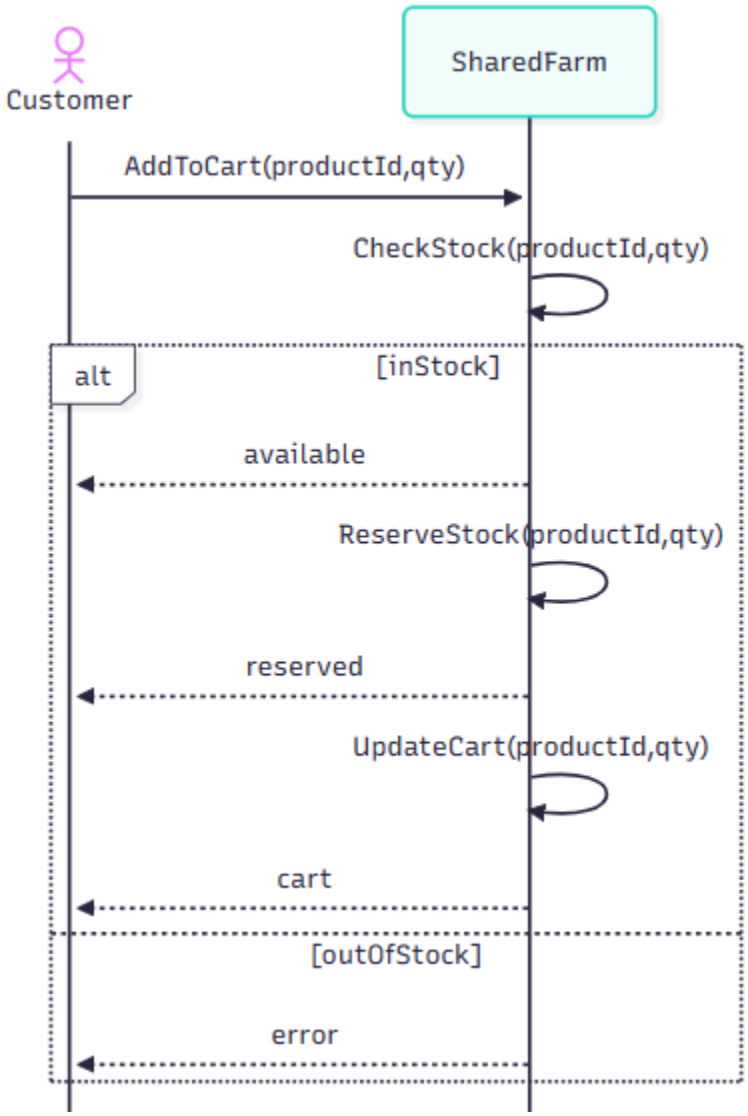
4. **View Product Details (Customer)**:

The actor opens a product page. The system loads the product and current stock. If found, it returns details and stock; if the product does not exist, it returns error.



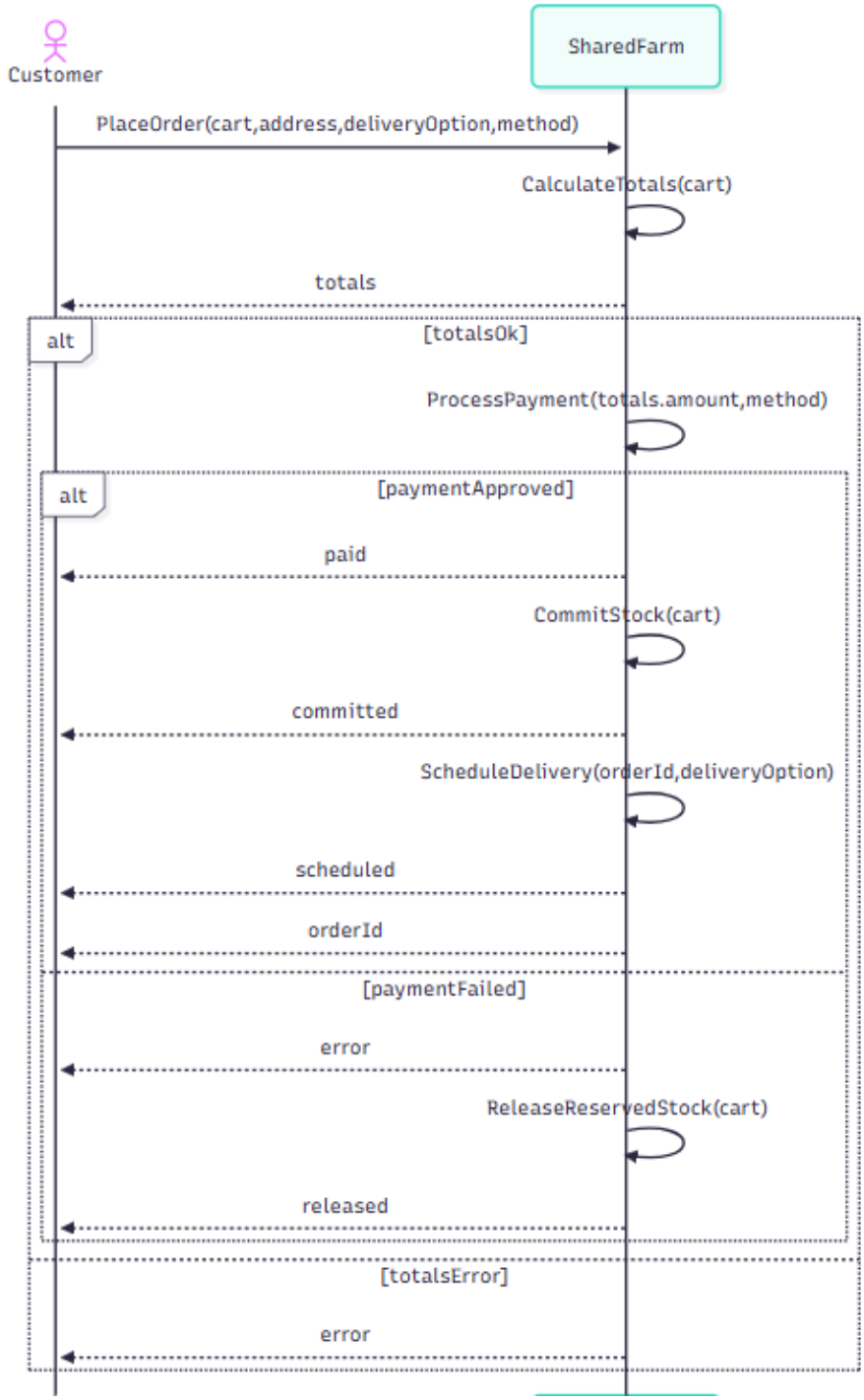
5. **Add Item to Cart (Customer)**:

The actor adds a product with a quantity. The system checks stock, reserves items, updates the cart, and returns the updated cart. If stock is insufficient, it returns error.



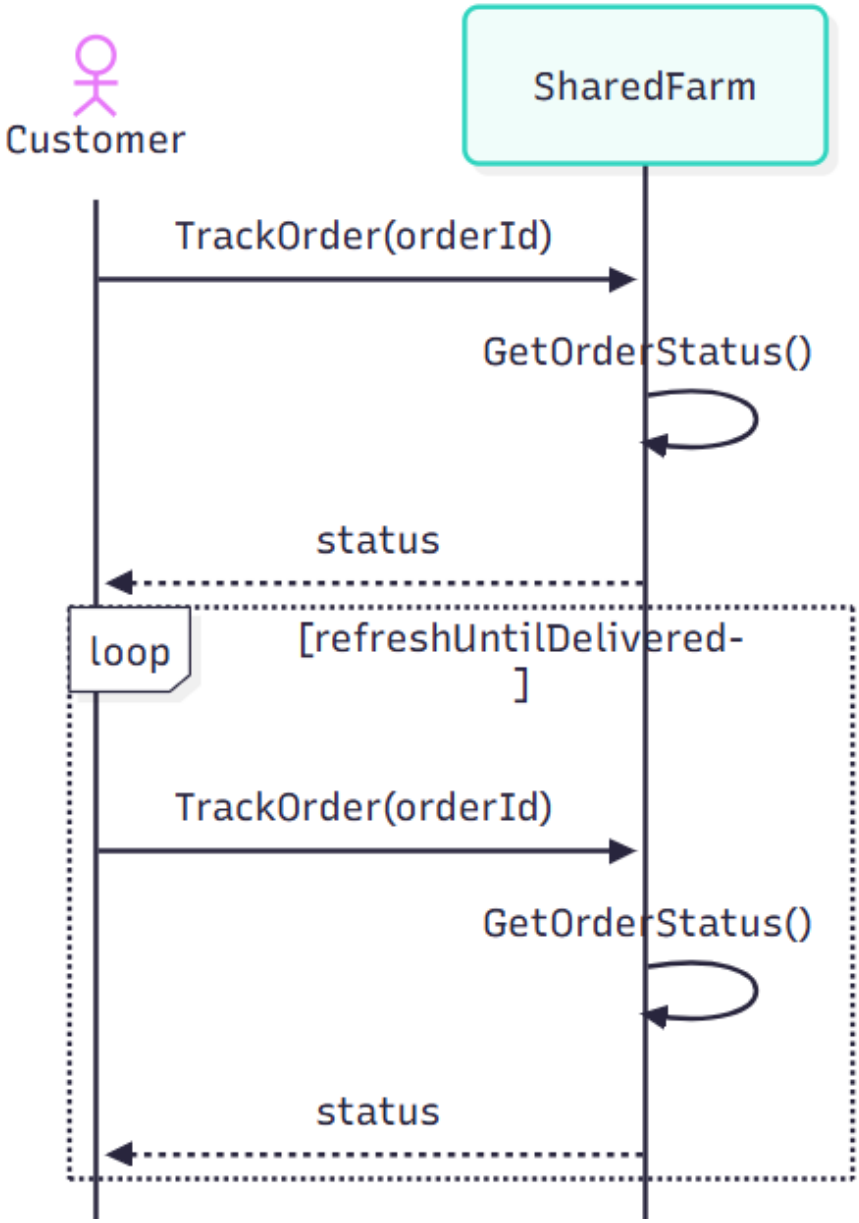
6. **Checkout Order (Customer)**:

The actor confirms checkout with address, delivery option, and payment method. The system calculates totals and processes the payment. If approved, it commits stock, books delivery, and returns an orderId. If totals are invalid or payment fails, it releases any holds and returns error.



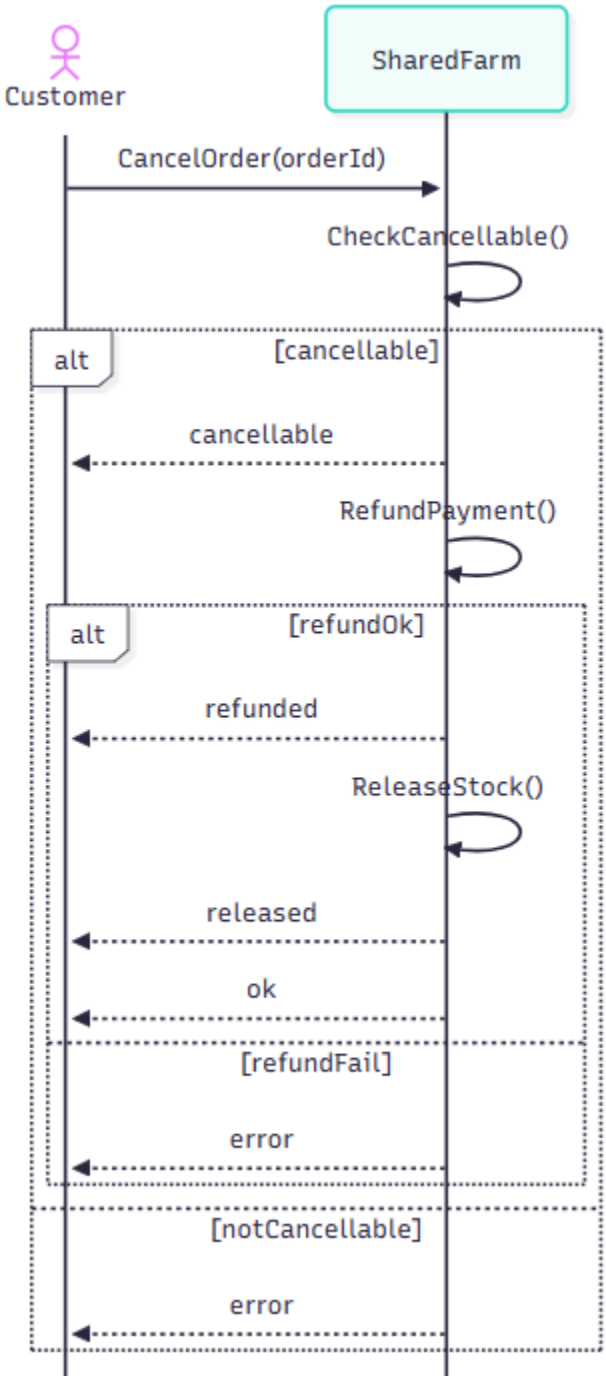
7. **Track Order (Customer)**:

The actor asks for the order status. The system fetches and returns the current status. The actor may repeat the request to refresh, receiving the latest status each time.



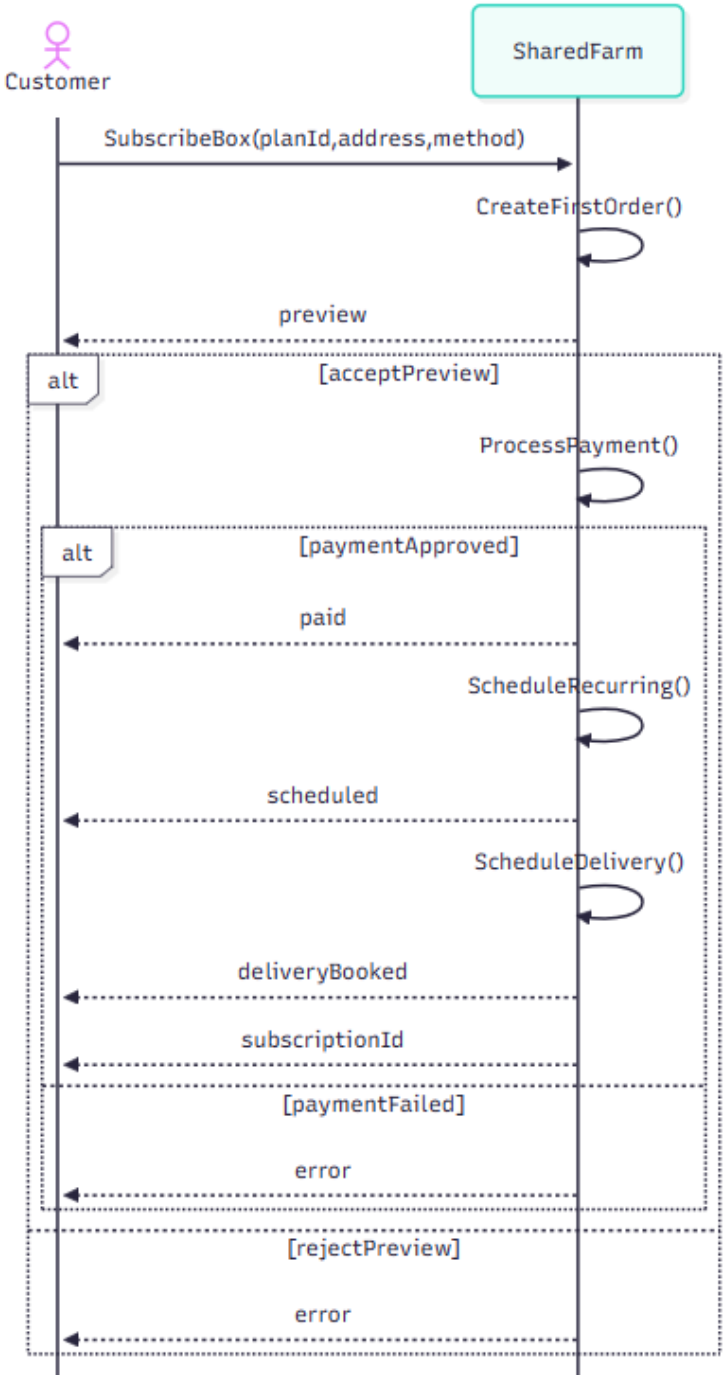
### 8. Cancel Order and Refund (Customer):

The actor requests cancellation. The system checks if the order is cancellable. If yes, it issues a refund, releases stock and returns ok (or refunded / released). If the order cannot be cancelled or the refund fails, it returns error.



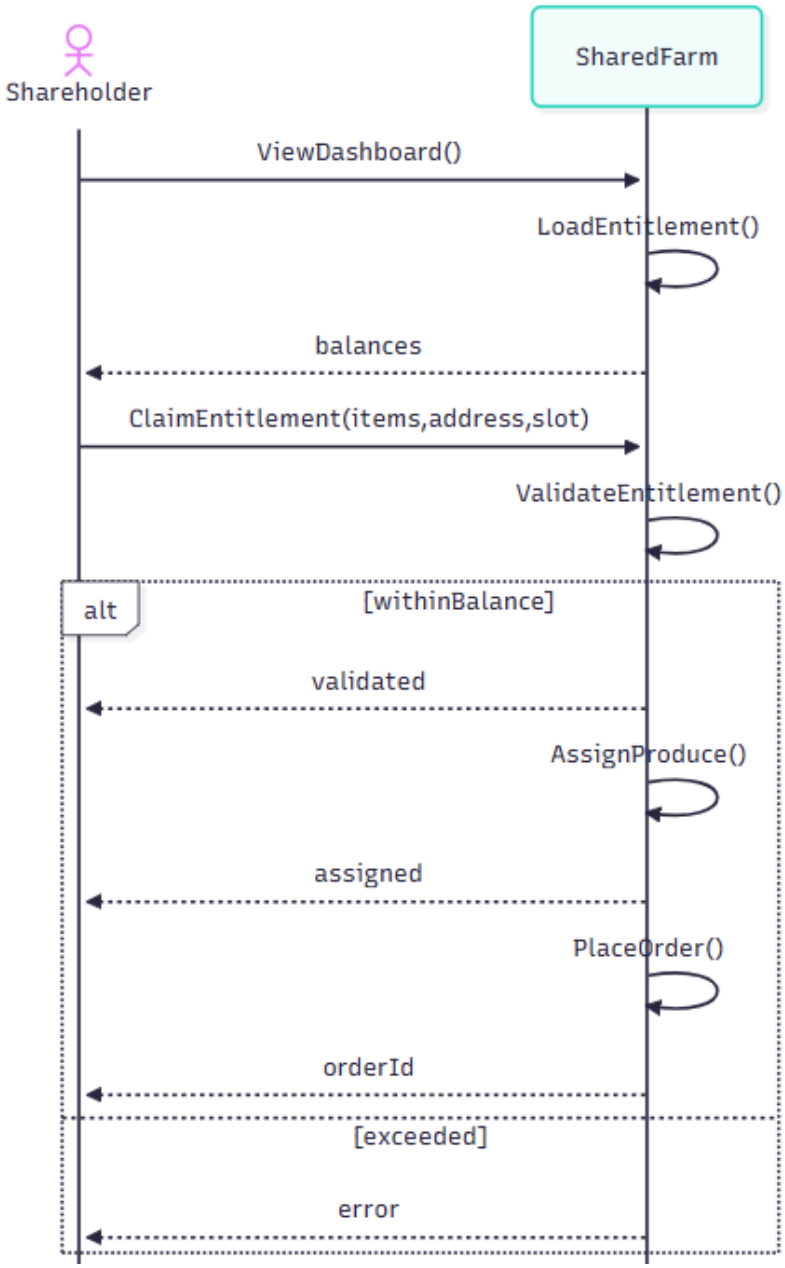
9. **Start Subscription (Customer)**:

The actor selects a subscription plan with address and payment method. The system builds a first-order preview. If the actor accepts and payment is approved, it schedules recurring deliveries and returns a subscription ID. If the actor rejects the preview or the payment fails, it returns error.



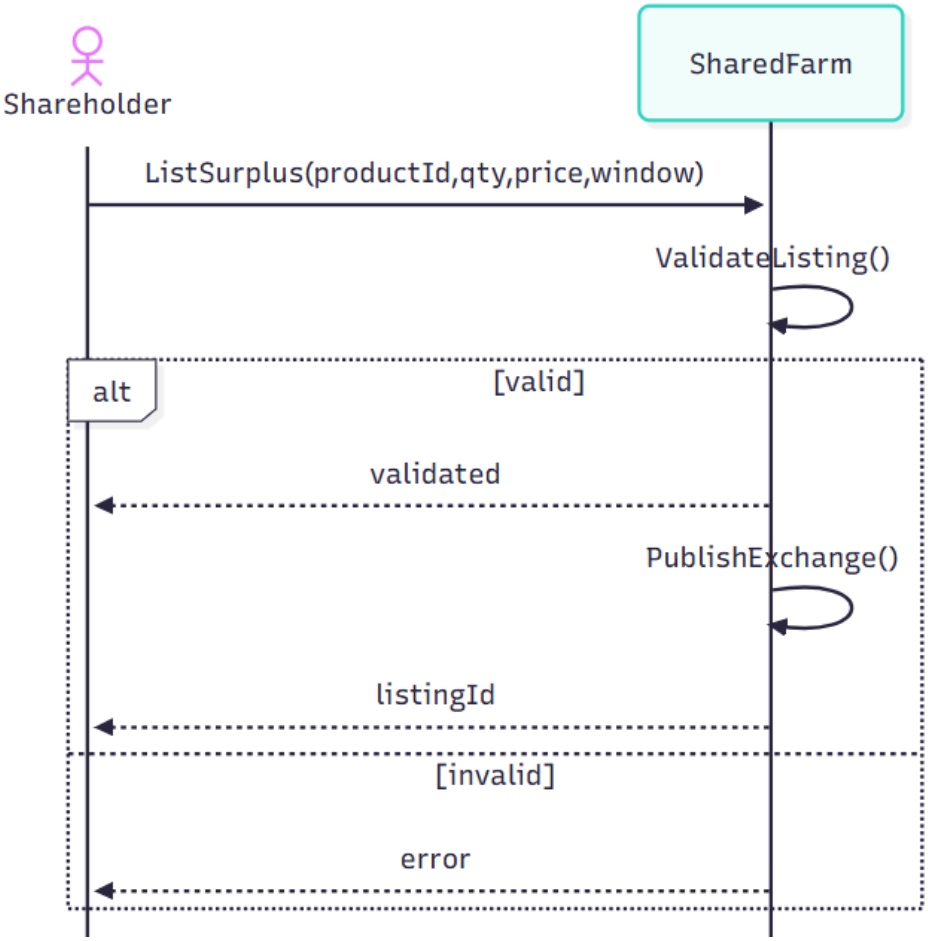
**10. Claim Produce Entitlement (Shareholder)**:

The actor opens the dashboard and submits an entitlement claim. The system loads balances, validates the requested items, assigns produce, creates a free order with a delivery slot, and returns an orderId. If the request exceeds the balance, it returns error.



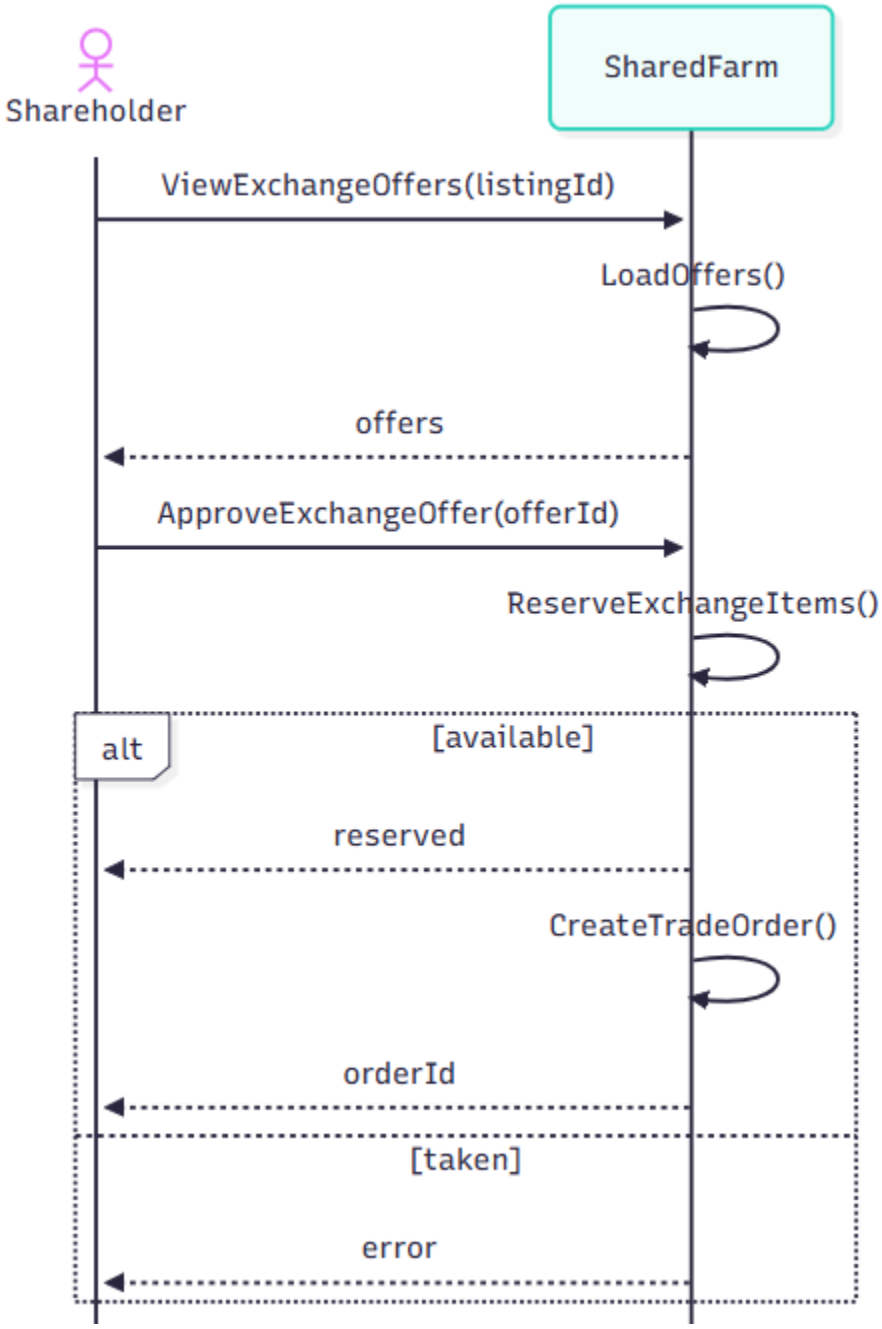
**11. List Surplus Produce (Shareholder)**:

The actor submits a surplus listing with product, quantity, price, and time window. The system validates the listing, publishes it to the exchange board, and returns a listingId. If validation fails, it returns error.



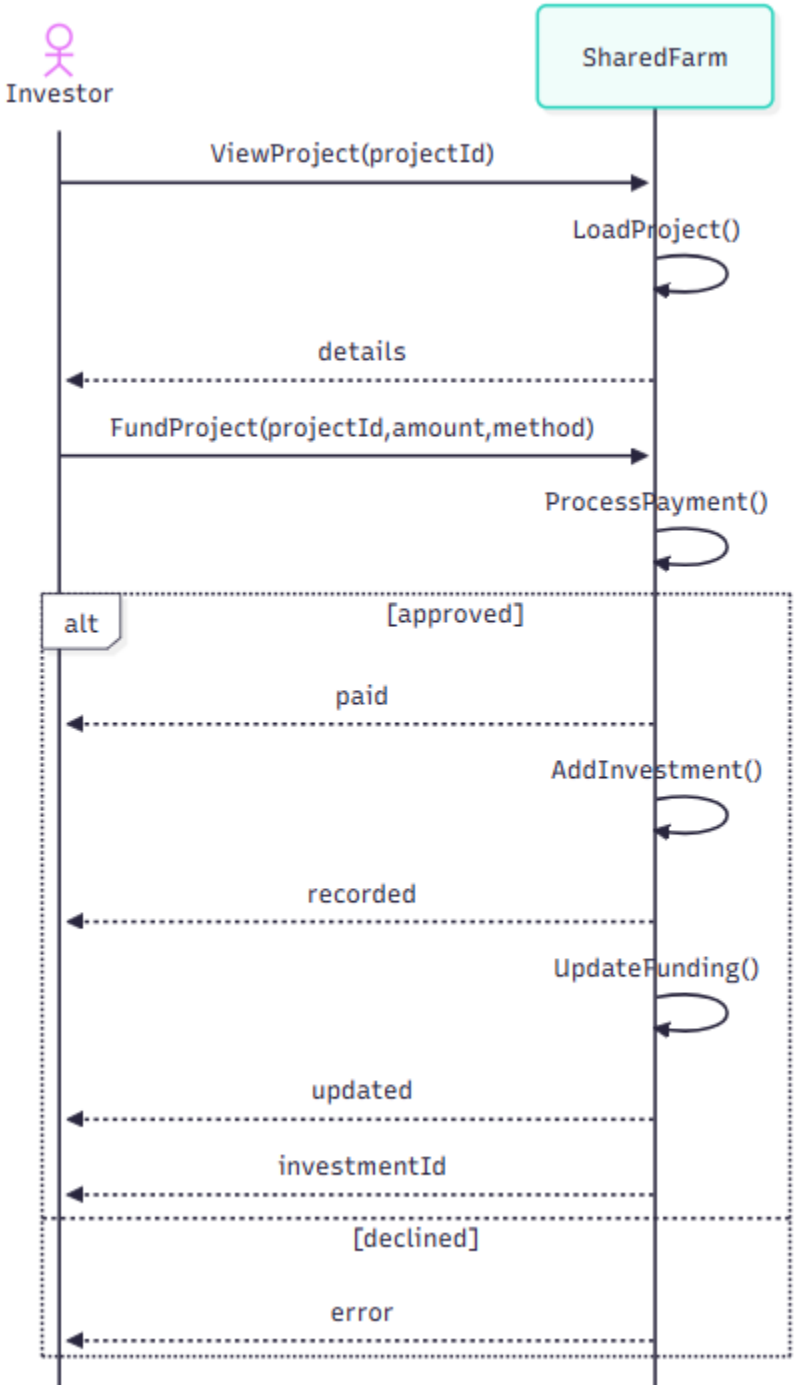
**12. Approve Exchange Offer (Shareholder)**:

The actor views offers for a listing and approves one. The system loads offers, reserves the items, creates a trade order, and returns an orderId. If the offer is already taken or reservation fails, it returns error.



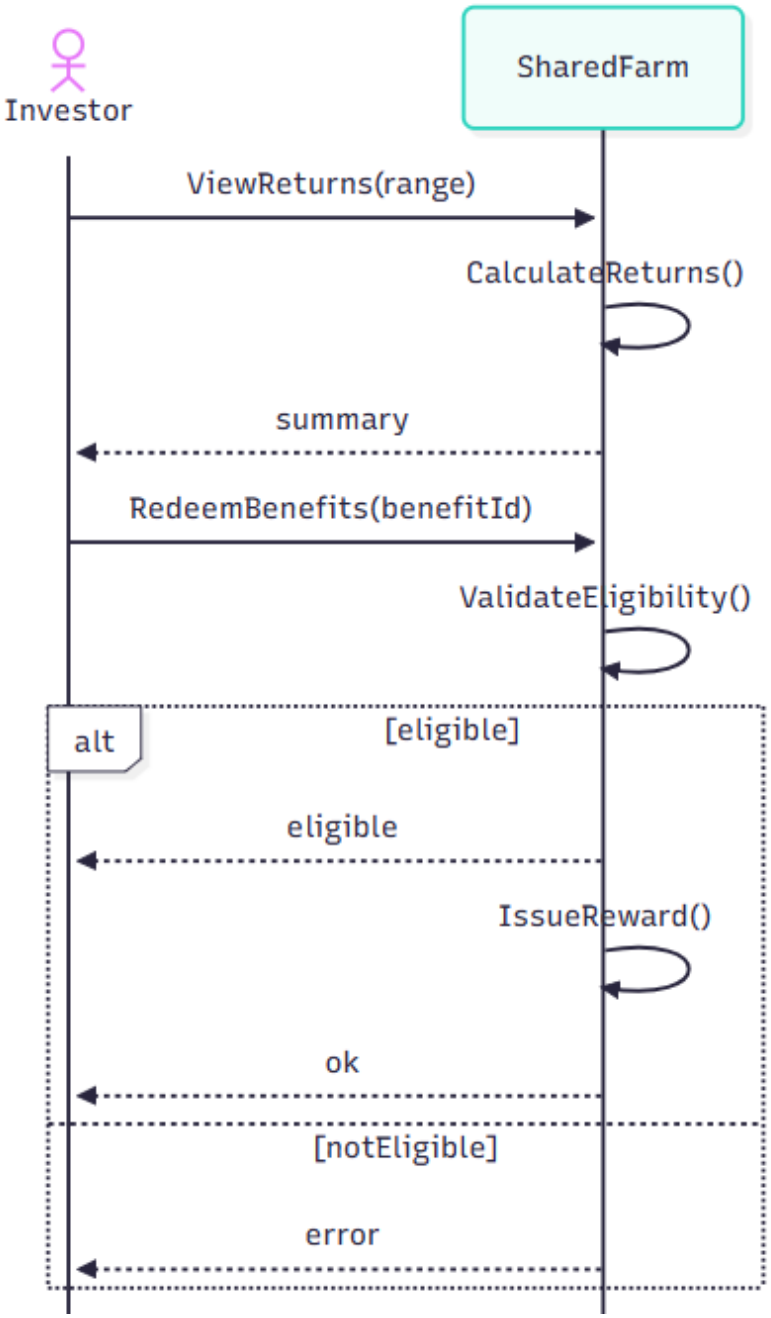
### 13. Fund Project (Investor)

The actor opens a project, then pledges an amount with a payment method. The system loads project details, processes payment, records the investment, updates funding progress, and returns an investmentId. If payment is declined, it returns error.



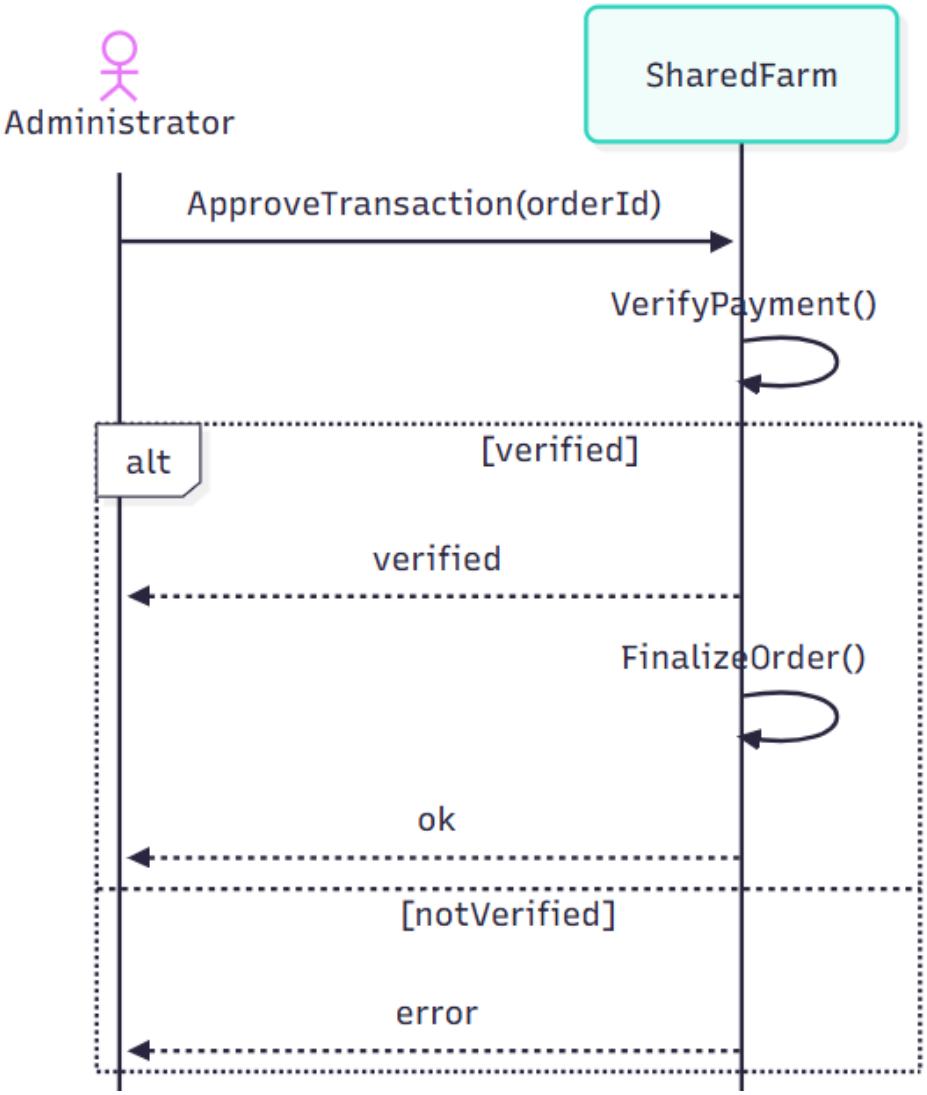
**14. Redeem Investment Benefits (Investor)**:

The actor views returns and requests to redeem a benefit. The system calculates the summary, validates eligibility, issues the reward, and returns ok. If not eligible, it returns error.



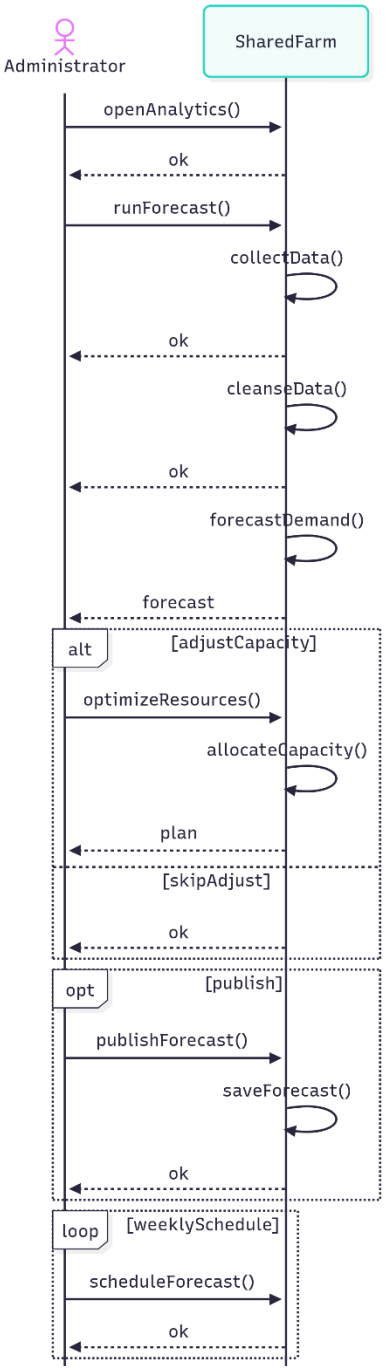
**15. Approve Transaction (Administrator)**:

The actor approves an order transaction. The system verifies payment, finalizes the order if verification succeeds, and returns ok. If payment cannot be verified, it returns error.



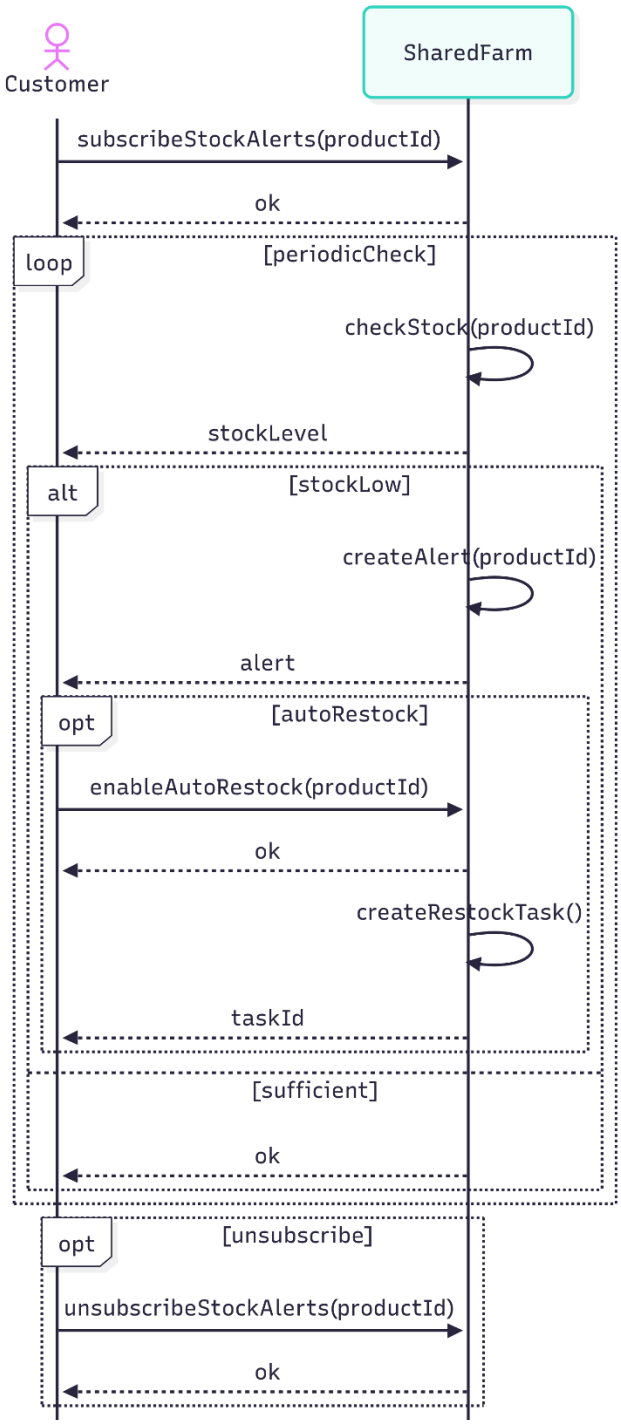
### 16. Demand Forecasting:

The actor opens analytics and runs a forecast. The system collects and cleans data, generates a demand forecast, and returns the forecast; if the actor chooses, it optimizes resources and saves the forecast. The system returns ok/plan on success; on any failure it returns error.



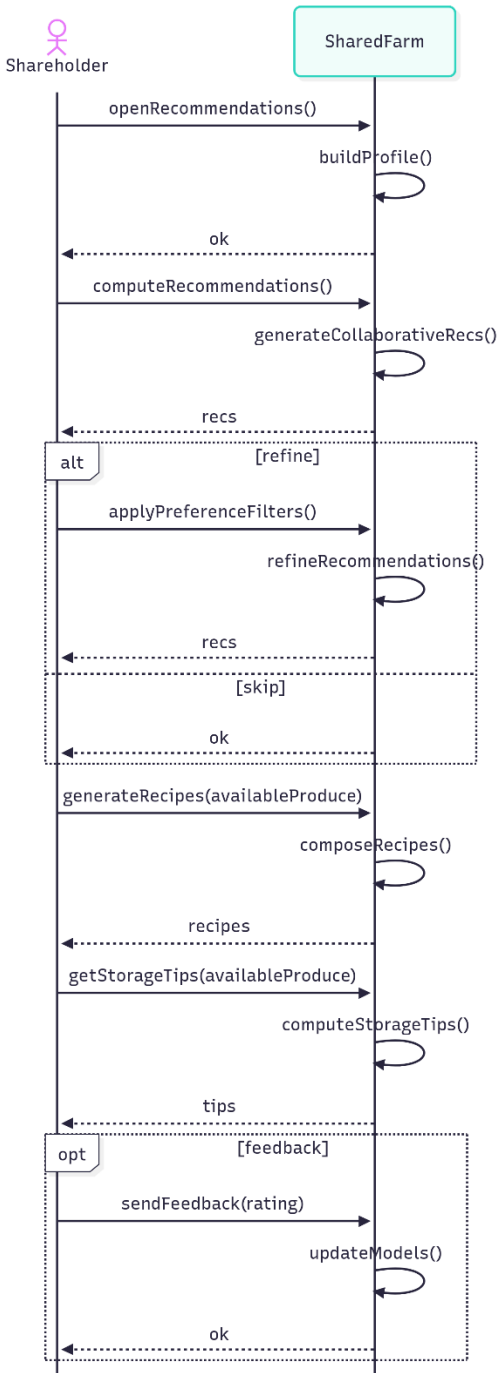
### 17. Stock & Order Automation (Alerts + Auto-Restock):

The actor subscribes to stock alerts. The system periodically checks stock; if low, it creates an alert and optionally creates an auto-restock task; if sufficient, it acknowledges. The system returns alert/task ID/ok on success; on invalid requests it returns error.



### 18. Personalized Recommendations:

The actor opens recommendations. The system builds a profile, returns recommendations, and on request generates recipes and storage tips; the actor may refine preferences and send feedback to update models. The system returns recs/recipes/tips/ok on success; if data is missing it returns error.



### 2.3 Special usage considerations

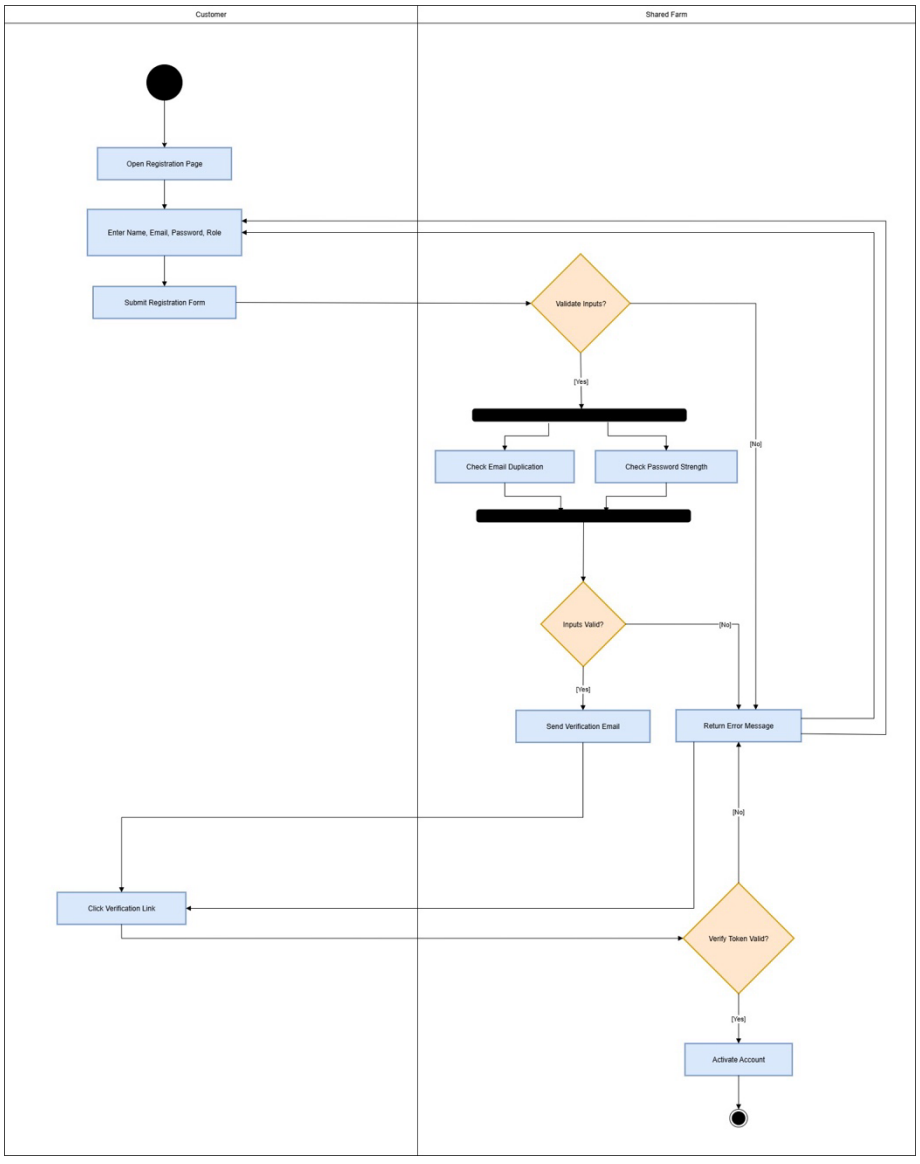
To guarantee appropriate use, the Shared Farm Web Application needs to be used with some additional requirements. Secure authentication and encrypted payment processing are essential because the platform deals with financial transactions. Because shareholders, consumers, investors, administrators, and community members all interact with the program in different ways, the system must also accommodate multiple user roles with varying levels of access. Transparency is also necessary; in order to preserve trust, users must be able to track the source of farm goods and look into reports. In order to accommodate both Arabic and English speakers in the United Arab Emirates, the system should also offer bilingual help.

### 2.4 Activity Diagrams

The graphical representation of the workflow of all use-cases may be presented.

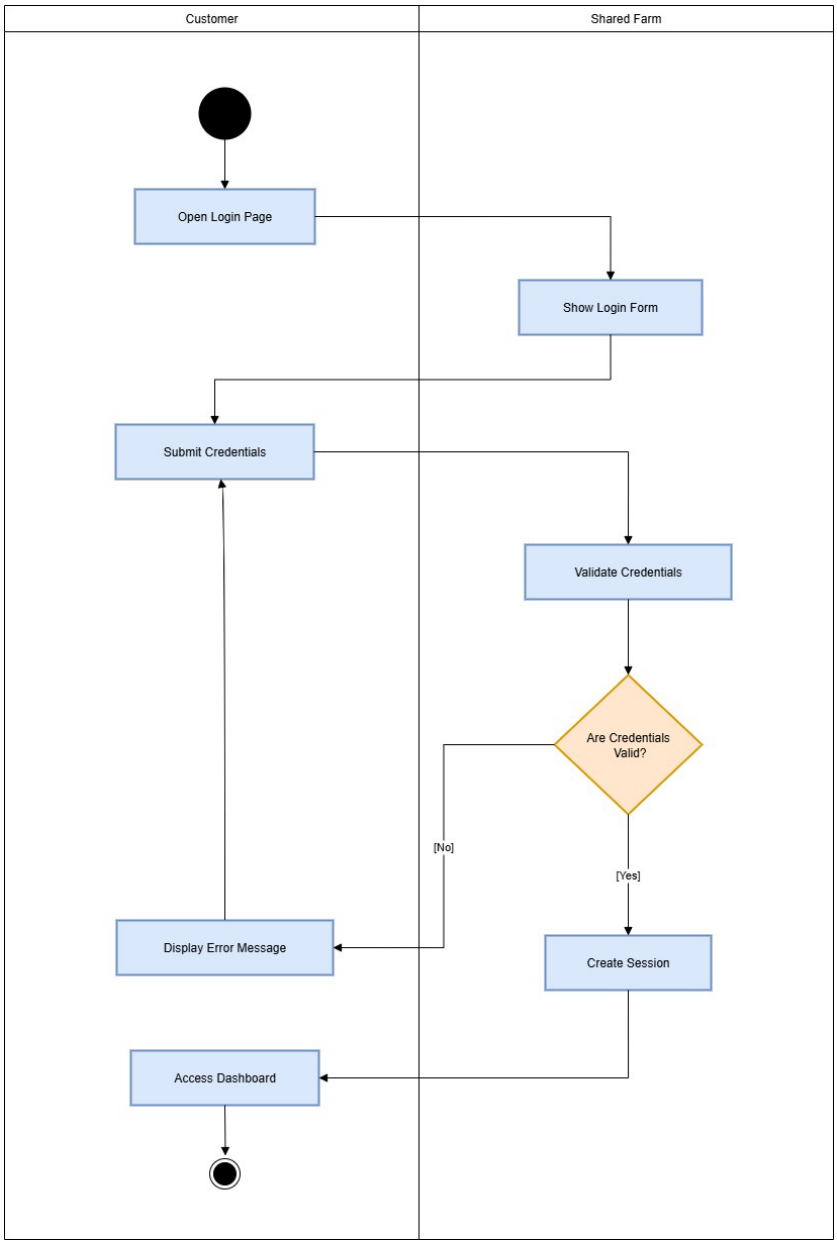
# Activity Diagrams:

1) **Register:**



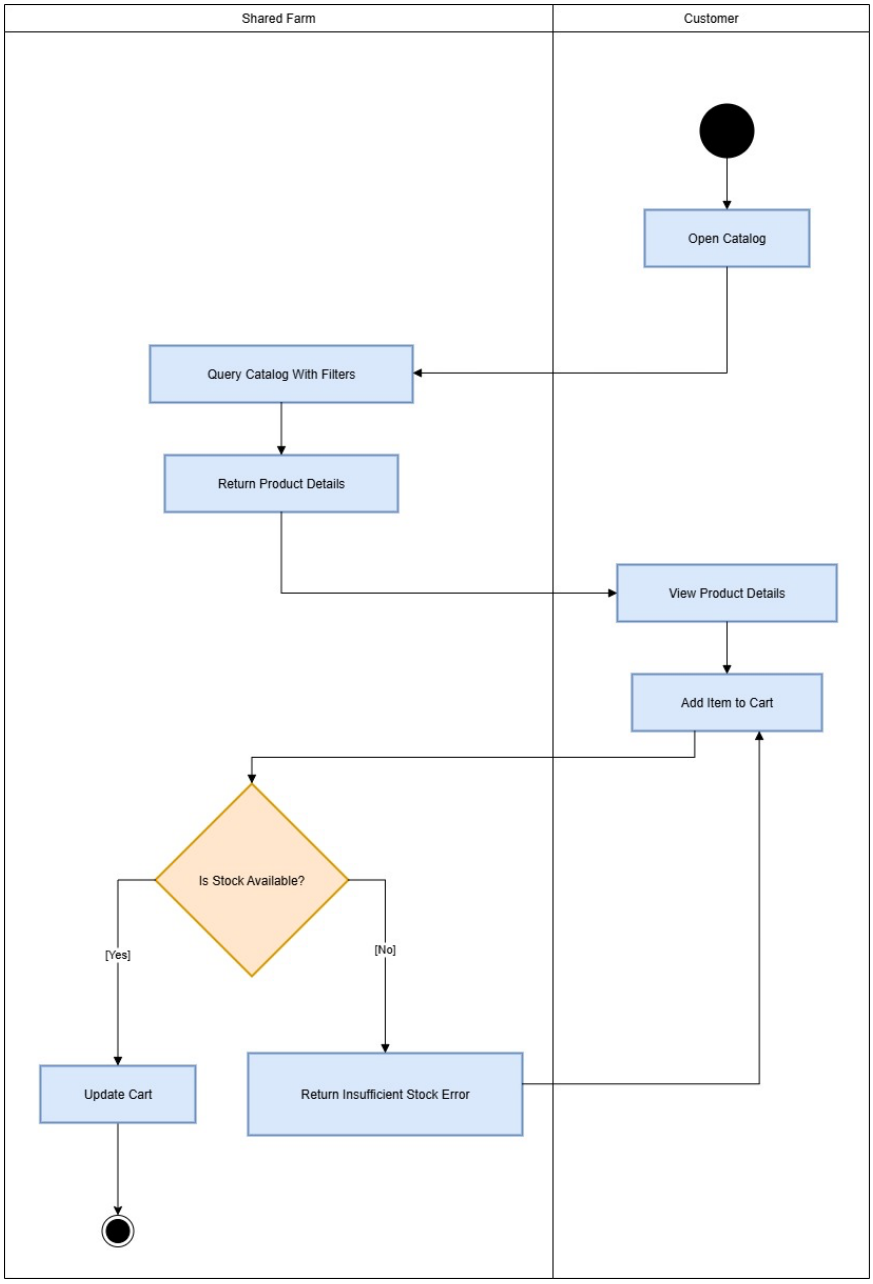
This diagram shows how a new user creates an account. The system validates the entered details, checks email and password, sends a verification email, and activates the account once the link is confirmed.

**2) Login:**



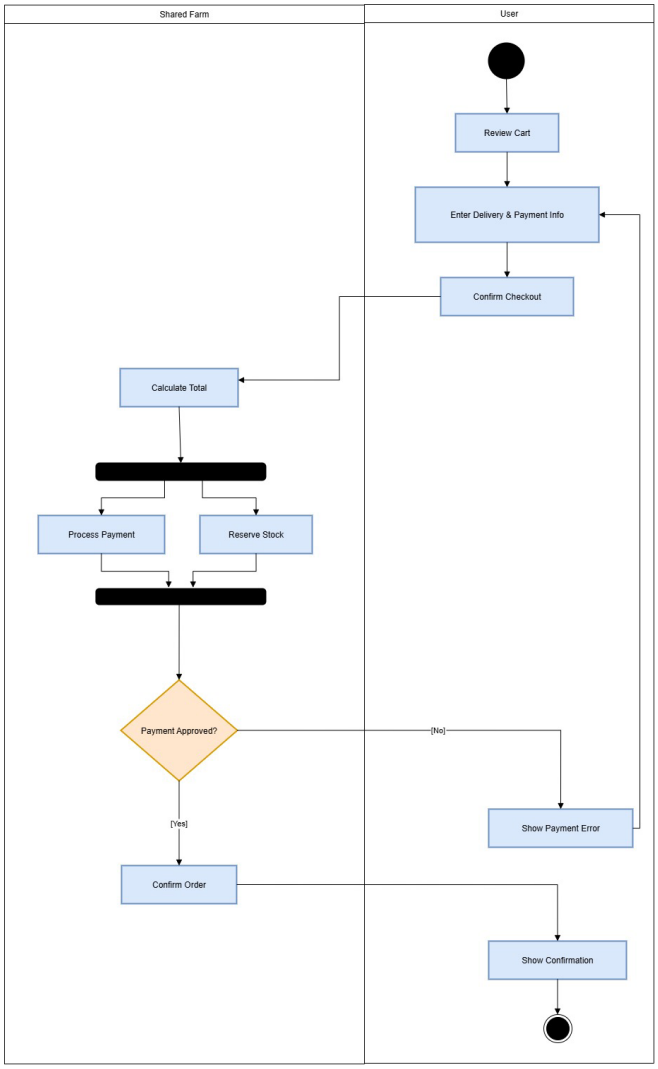

This diagram shows how a user logs into the system. The user submits credentials, the system validates them, and access is granted if valid; otherwise, an error message is displayed.

### 3) Browse Catalogue:



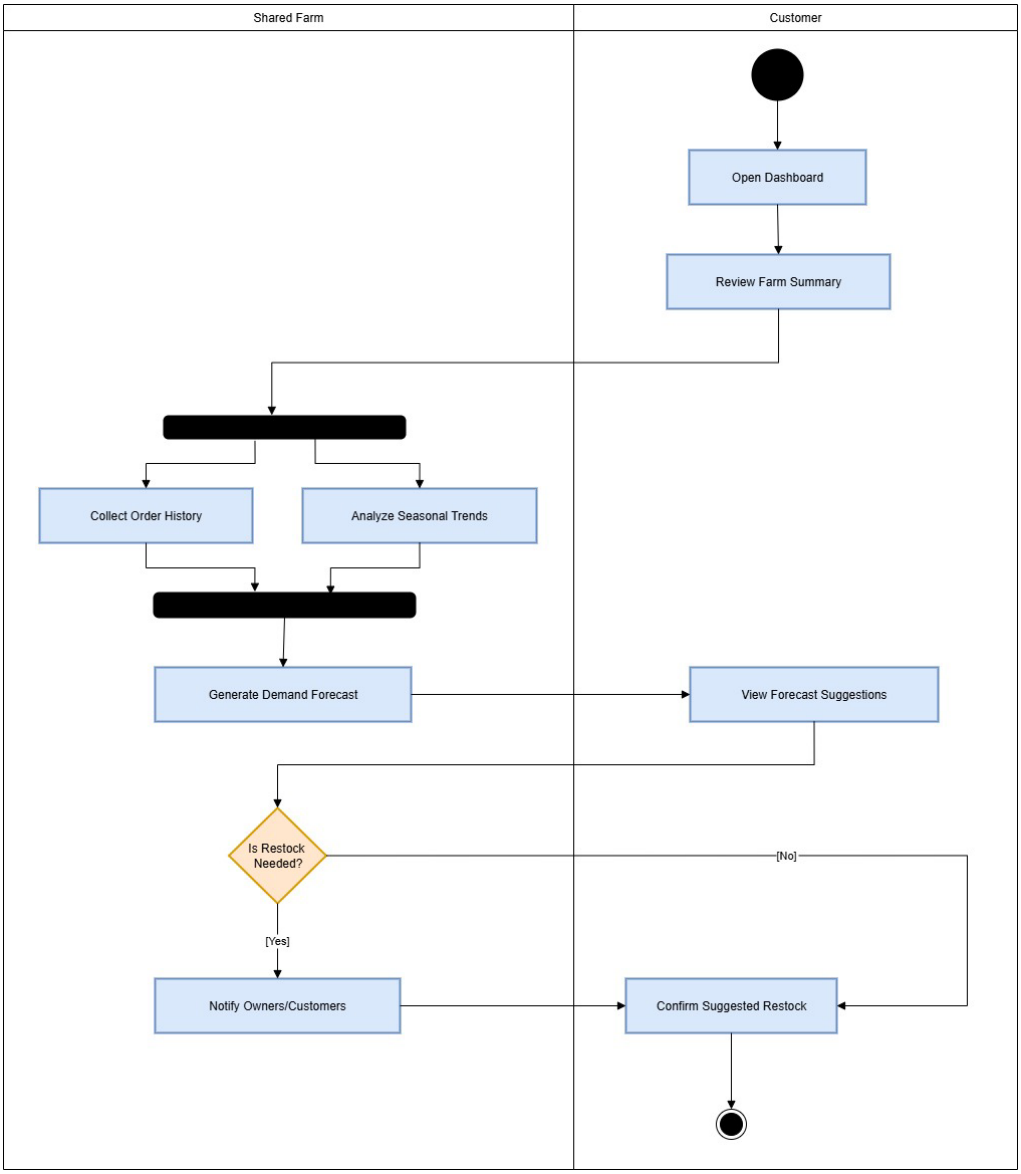

This diagram shows how a customer browses the catalog, views product details, and adds items to the cart. The system checks stock availability and either updates the cart or returns an error if stock is insufficient.

## 4) Checkout order:



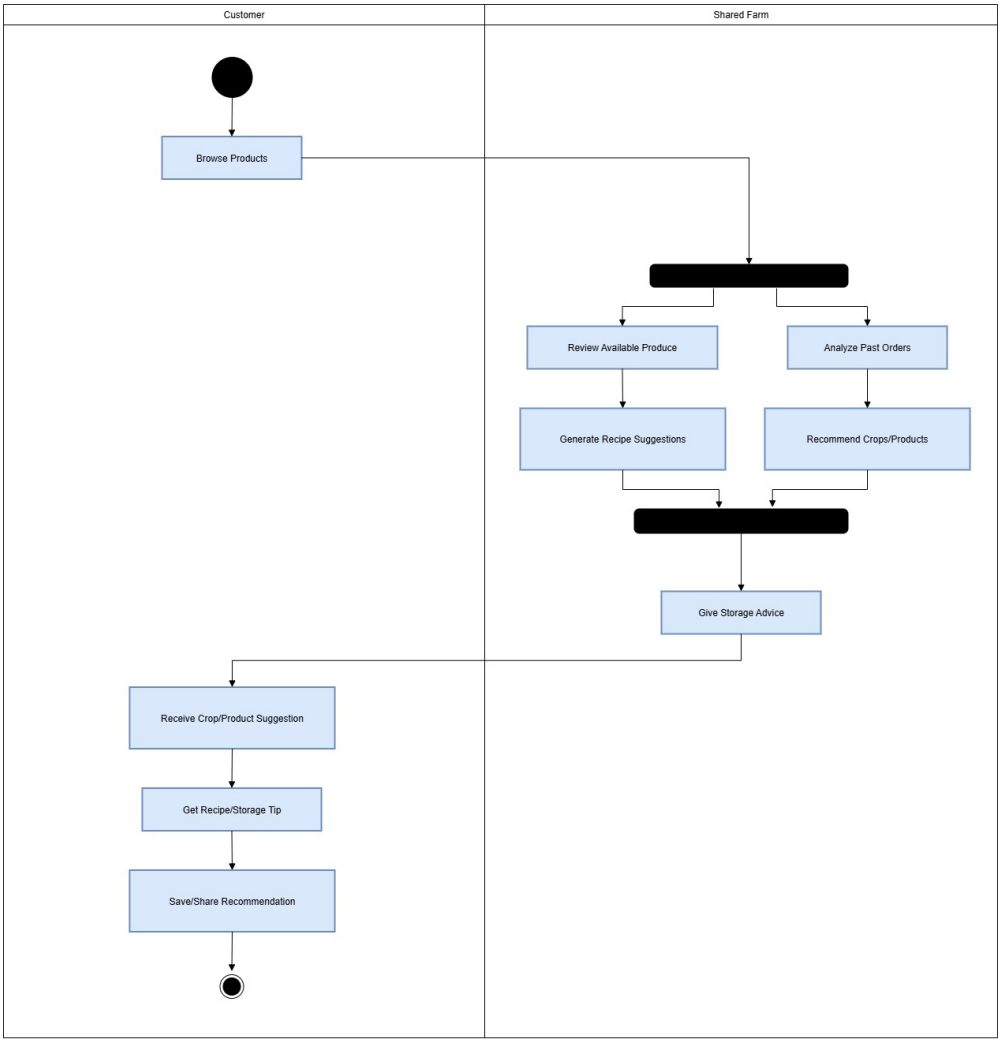

This diagram shows how a user confirms checkout by entering delivery and payment details. The system calculates the total, processes payment, and reserves stock. If payment is approved, the order is confirmed; otherwise, a payment error is displayed.

## 5) Demand Forecasting (AI-assisted):




This diagram shows how the system analyzes order history and seasonal trends to generate demand forecasts. Users can view forecast suggestions, and if restocking is required, owners and customers are notified to confirm the suggested restock.

## 6) Personalized Recommendations & Recipe Suggestions:




This diagram shows how the system reviews available produce and past orders to generate recipe suggestions, recommend products, and provide storage advice. Customers receive tailored tips that can be saved or shared.

## 3.0 Data Model and Description

This section describes the main objects in the Shared Farm Web Application and the relationships between them

### 3.1 Data objects

This section describes the main data objects in the Shared Farm Web Application and their key attributes and methods.

Class name: Profile
• Major attributes: profileId, avatar, preferences
• Major methods: SetAvatar(), SetPreferences()

Class name: Customer
• Major attributes: memberSince, loyaltyPoints
• Major methods: BrowseCatalog(), FilterProducts(), ViewProductDetails(), AddItemToCart(), UpdateCartQuantity(), ApplyPromoCode(), CheckoutOrder(), TrackOrder(), CancelOrder(), RequestRefund(), RateProduct()

Class name: Shareholder
• Major attributes: shareholderId, unitsOwned
• Major methods: ViewDashboard(), VoteOnDecision(), ClaimProduceEntitlement(), TransferEntitlement(), TrackEntitlementDelivery(), ApproveExchangeOffer(), DownloadOwnerStatement()

Class name: Investor
• Major attributes: investorId, riskProfile
• Major methods: BrowseProjects(), ViewProjectDetails(), FundProject(), ViewPortfolio(), TrackReturns(), RedeemInvestmentBenefits(), DownloadInvestmentReceipt()

Class name: Cart
• Major attributes: cartId, totalItems, totalAmount
• Major methods: UpdateCart(), ClearCart()

Class name: CartItem
• Major attributes: cartItemId, qty, price
• Major methods: SetQuantity(), SetLineTotal()

Class name: PromoCode
• Major attributes: code, discount, status
• Major methods: ValidatePromo(), ApplyPromo()

Class name: Review
• Major attributes: reviewId, rating, comment
• Major methods: AddReview(), RateProduct()

Class name: Catalog
• Major attributes: catalogId, title
• Major methods: QueryCatalog(), AddProduct(), UpdateProduct(), RemoveProduct()

Class name: Product
• Major attributes: productId, name, category, price, quantity
• Major methods: ViewDetails(), CalculatePrice(), UpdateStock()

Class name: CropProduct
• Major attributes: grade
• Major methods: WashGradePack()

Class name: AnimalProduct
• Major attributes: processNote
• Major methods: PasteurizeOrBleed()

Class name: Inventory
• Major attributes: inventoryId, total, reserved, available
• Major methods: CheckStock(), ReserveStock(), ReleaseReservedStock(), CommitStock(), UpdateInventory()

Class name: Order
• Major attributes: orderId, total, createdAt
• Major methods: CalculateTotal(), CreateOrder(), GetOrderStatus(), CheckCancellable(), CancelOrder()

Class name: OrderLine
• Major attributes: orderLineId, qty, unitPrice, lineTotal
• Major methods: SetLineTotal()

Class name: OrderStatus
• Major attributes: state, timestamp
• Major methods: SetCreated(), SetConfirmed(), SetPacked(), SetShipped(), SetDelivered(), SetCancelled(), SetRefunded()

Class name: Payment
• Major attributes: paymentId, amount, currency
• Major methods: ProcessPayment(), VerifyPayment(), RefundPayment()

Class name: PaymentMethod
• Major attributes: methodId, provider, isAd
• Major methods: SelectMethod(), VerifyPaymentMethod()

Class name: PaymentStatus
• Major attributes: state, updatedAt
• Major methods: SetPending(), SetAuthorized(), SetFailed(), UpdateStatus()

Class name: PaymentGateway
• Major attributes: gatewayId, endpoint
• Major methods: AuthenticateTransaction(), VerifyPaymentMethod(), ProcessPayment(), RefundTransaction()

Class name: Delivery
• Major attributes: deliveryId, addressRef, slot
• Major methods: ScheduleDelivery(), UpdateDeliveryStatus(), ConfirmDeliveryCompletion(), ReportDeliveryIssue()

Class name: DeliveryStatus
• Major attributes: state, updatedAt
• Major methods: SetScheduled(), SetInTransit(), SetDelivered(), SetFailed(), SetCancelled()

Class name: DeliveryPartner
• Major attributes: partnerId, name
• Major methods: AcceptDeliveryRequest(), UpdateDeliveryStatus(), ConfirmDeliveryCompletion(), ReportDeliveryIssue()

Class name: OwnerStatement
• Major attributes: statementId, period
• Major methods: GenerateOwnerStatement(), DownloadOwnerStatement()

Class name: Entitlement
• Major attributes: entitlementId, balance, period
• Major methods: LoadEntitlement(), ValidateEntitlement(), AssignProduce(), TransferEntitlement()

Class name: SurplusListing
• Major attributes: listingId, qty, price, window
• Major methods: ValidateListing(), PublishExchange()

Class name: ExchangeOffer
• Major attributes: offerId, offeredQty, status
• Major methods: LoadOffer(), ApproveExchangeOffer(), AcceptExchangeOffer(), ReserveExchangeItems(), CreateTradeOrder()

Class name: TradeOrder
• Major attributes: tradeOrderId, total
• Major methods: SettleTrade()

Class name: Reward
• Major attributes: rewardId, kind
• Major methods: RedeemInvestmentBenefit()

Class name: Investment
• Major attributes: investmentId, amount
• Major methods: AddInvestment(), CalculateReturns(), ValidateEligibility(), IssueReward()

Class name: InvestmentReceipt
• Major attributes: receiptId
• Major methods: GenerateInvestmentReceipt(), DownloadInvestmentReceipt()

Class name: Portfolio
• Major attributes: portfolioId, summary
• Major methods: ViewPortfolio()

Class name: LearningContent
• Major attributes: contentId, title
• Major methods: AccessLearningContent()

Class name: Event
• Major attributes: eventId, title, date
• Major methods: BrowseEvents(), RegisterForWorkshop(), CancelWorkshopRegistration(), SubmitEventFeedback()

Class name: BlogPost
• Major attributes: postId, title, status
• Major methods: ShareBlogPost(), ModerateContent()

Class name: Report
• Major attributes: reportId, period
• Major methods: GenerateSalesReport(), GenerateImpactReport(), ExportReport()

Class name: AuditLog
• Major attributes: logId, actor
• Major methods: AddLog(), ViewLog(), GenerateSummary()

Class name: UserAccount
• Major attributes: userId, name, email, passwordHash, role, status
• Major methods: Register(), Login(), ValidateCredentials(), CreateUser(), CreateSession(), EnableRememberMe(), UpdateProfile(), DeleteAccount()

Class name: Administrator
• Major attributes: adminId, accessLevel
• Major methods: ManageUsers(), ManageCatalog(), ApproveListing(), ApproveTransaction(), VerifyPayment(), IssueRefund(), GenerateReports(), ConfigureSettings(), ModerateContent()

Class name: CommunityMember
• Major attributes: handle, reputation

• Major methods: AccessLearningContent(), BrowseEvents(), RegisterForWorkshop(), CancelWorkshopRegistration(), SubmitEventFeedback(), ShareBlogPost()

Class name: Project
• Major attributes: projectId, title, goal, raised
• Major methods: LoadProject(), UpdateFunding(), CloseWhenFunded()

### 3.2 Relationships

Below are the relationships of our objects:

● Customer ‘has a’ Profile (association)
● Customer ‘has a’ Cart (association)
● Cart ‘has a’ CartItem (association)
● Cart ‘has a’ PromoCode (association)
● Cart ‘has a’ Review (association)
● Customer ‘is a’ UserAccount (inheritance)
● Shareholder ‘is a’ UserAccount (inheritance)
● Investor ‘is a’ UserAccount (inheritance)
● Administrator ‘is a’ UserAccount (inheritance)
● CommunityMember ‘is a’ UserAccount (inheritance)

● Shareholder ‘has a’ OwnerStatement (association)
● OwnerStatement “strongly part of” Shareholder (composition)

● Shareholder ‘has a’ Entitlement (association)
● Entitlement “strongly part of” Shareholder (composition)

● Entitlement ‘has a’ SurplusListing (association)
● SurplusListing ‘has a’ ExchangeOffer (association)
● ExchangeOffer ‘has a’ TradeOrder (association)
● TradeOrder “strongly part of” ExchangeOffer (composition)

● Investment ‘has a’ InvestmentReceipt (association)
● Investment ‘has a’ Reward (association)
● InvestmentReceipt “strongly part of” Investment (composition)
● Reward “part of” Investment (aggregation)

● Catalog ‘has a’ Product (association)
● Product ‘has a’ Inventory (association)
● Product “is a” CropProduct (inheritance)
● Product “is a” AnimalProduct (inheritance)

● Order ‘has a’ OrderLine (association)
● OrderLine “strongly part of” Order (composition)
● Order ‘has a’ OrderStatus (association)

● Order ‘has a’ Payment (association)
● Payment ‘has a’ PaymentMethod (association)
● Payment ‘has a’ PaymentStatus (association)
● Payment ‘has a’ PaymentGateway (association)
● PaymentStatus “strongly part of” Payment (composition)

● Delivery ‘has a’ DeliveryStatus (association)
● Delivery ‘has a’ DeliveryPartner (association)
● DeliveryStatus “strongly part of” Delivery (composition)

● CommunityMember ‘has a’ Portfolio (association)
● Portfolio ‘has a’ Investment (association)
● Portfolio “strongly part of” Investor (composition)

● CommunityMember ‘has a’ LearningContent (association)
● CommunityMember ‘has a’ Event (association)
● CommunityMember ‘has a’ BlogPost (association)
● CommunityMember ‘has a’ Report (association)

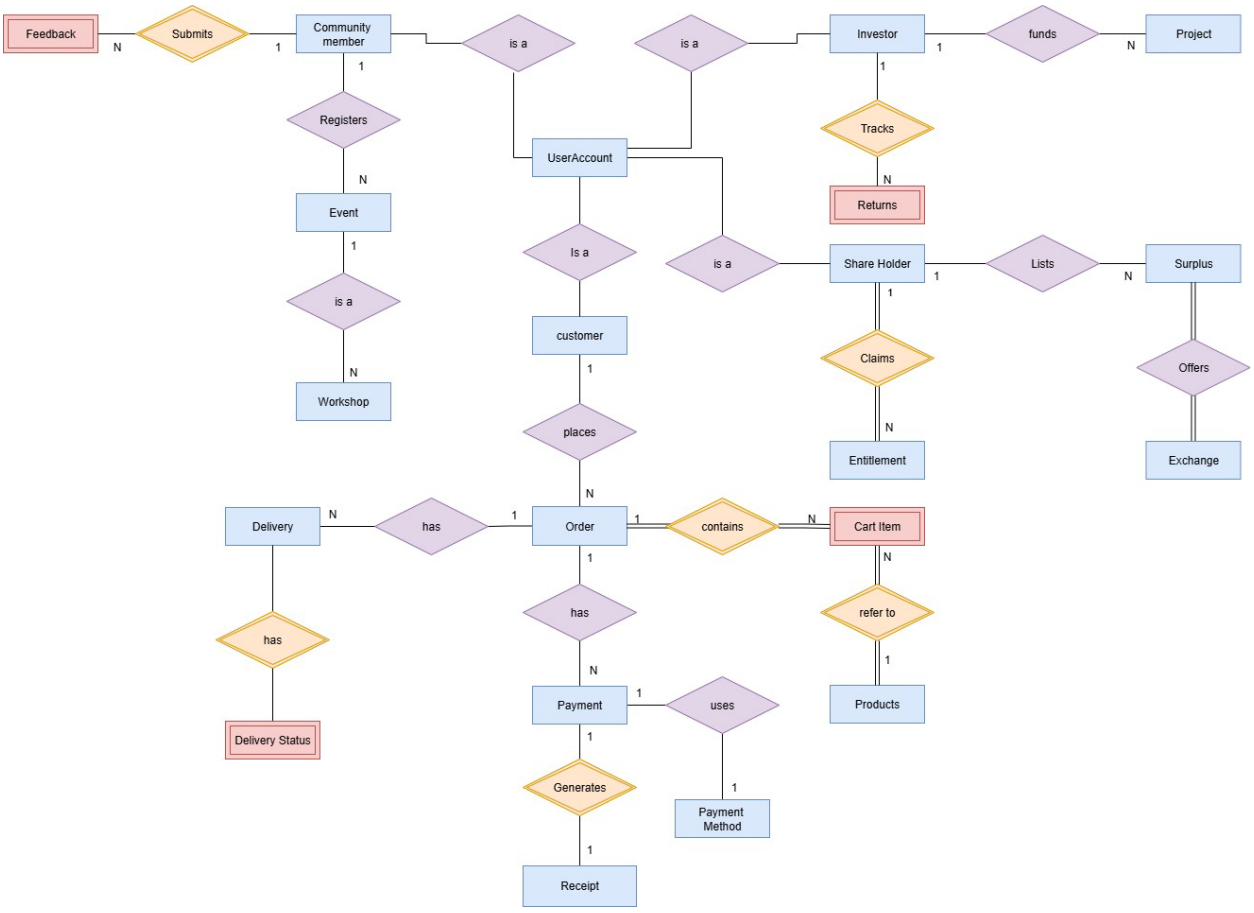
● Administrator ‘has a’ AuditLog (association)
● AuditLog “strongly part of” Administrator (composition)

● Project ‘has a’ Investor (association)
● Project “strongly part of” Investment (composition)

### 3.3 Complete data model

Use UML class diagrams to develop an ERD-like model of the data objects and relationships.

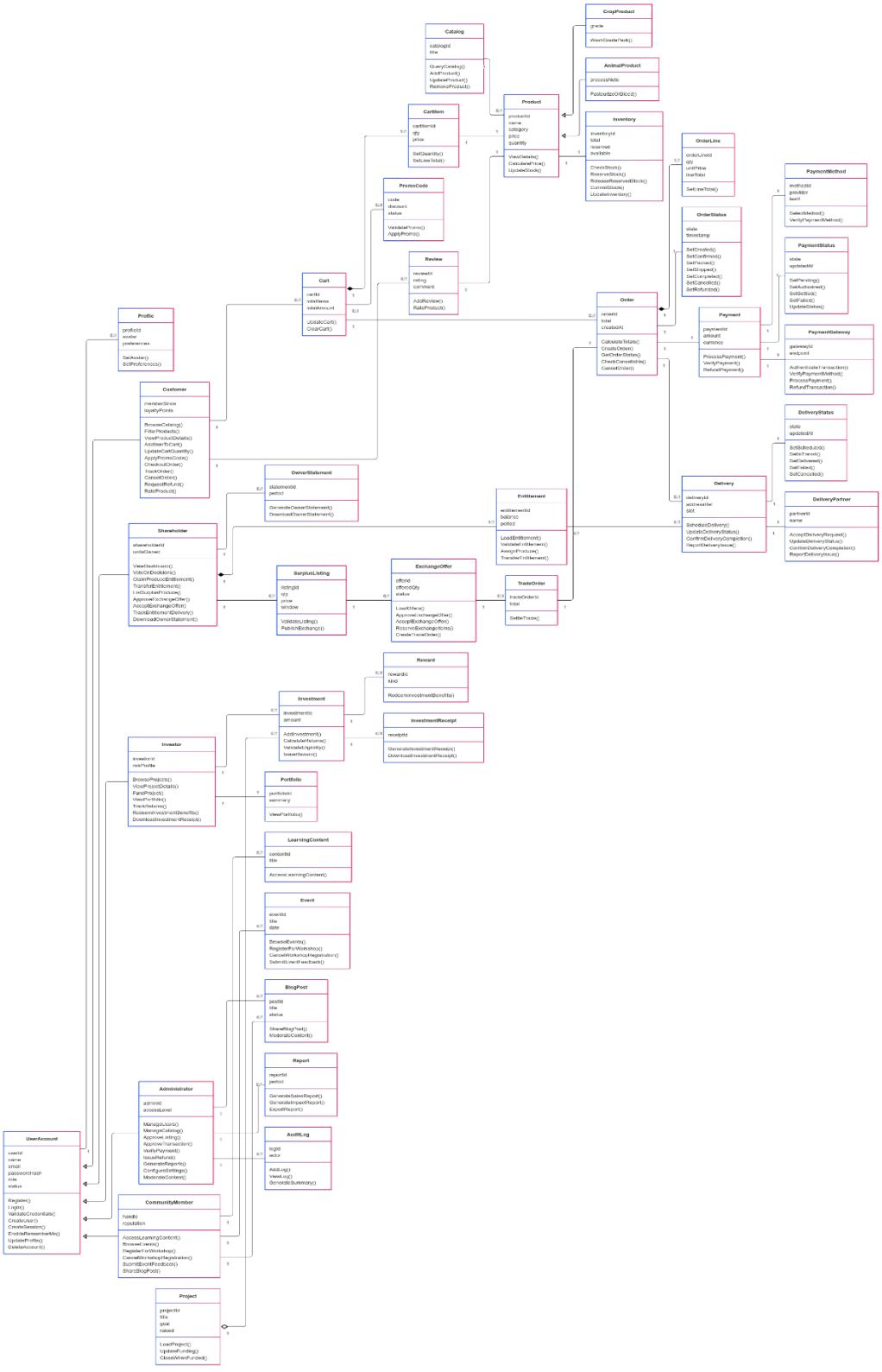
# Complete Data Model:



## 4.0 Functional Model and Description

This section describes the static structure of the software.

### 4.1 Class diagrams



## 4.2 Software Interface Description

The Shared Farm Web Application showcases an intuitive interface aimed at making farm ownership and product buying easy for every user. Straightforward navigation menus enable shareholders, customers, and investors to engage effortlessly with the system, whether purchasing produce, subscribing to boxes, or financing projects. Linking with secure payment gateways guarantees safe transactions, while alerts inform users about orders, deliveries, and investment progress. The design will adapt seamlessly to both desktop and mobile devices, ensuring a fluid experience in each setting

### 4.2.1 External machine interfaces

The system will operate on standard web browsers across desktops, laptops, and smartphones. Mobile devices might additionally allow push notifications for information on orders and deliveries

### 4.2.2 External system interfaces

User Authentication and Access Control: Safe entry with various roles (shareholder, client, investor, administrator).

Payment Gateways: Incorporation with UAE-compatible payment APIs (credit cards, debit cards, digital wallets).

Cloud Services: Outsourced cloud storage will securely handle product, order, and investment information.

Messaging and Alerts: Incorporation for system notifications, order confirmations, and reminders.

AI Engine: Connects with forecasting and suggestion modules for demand forecasting and recipe recommendations.

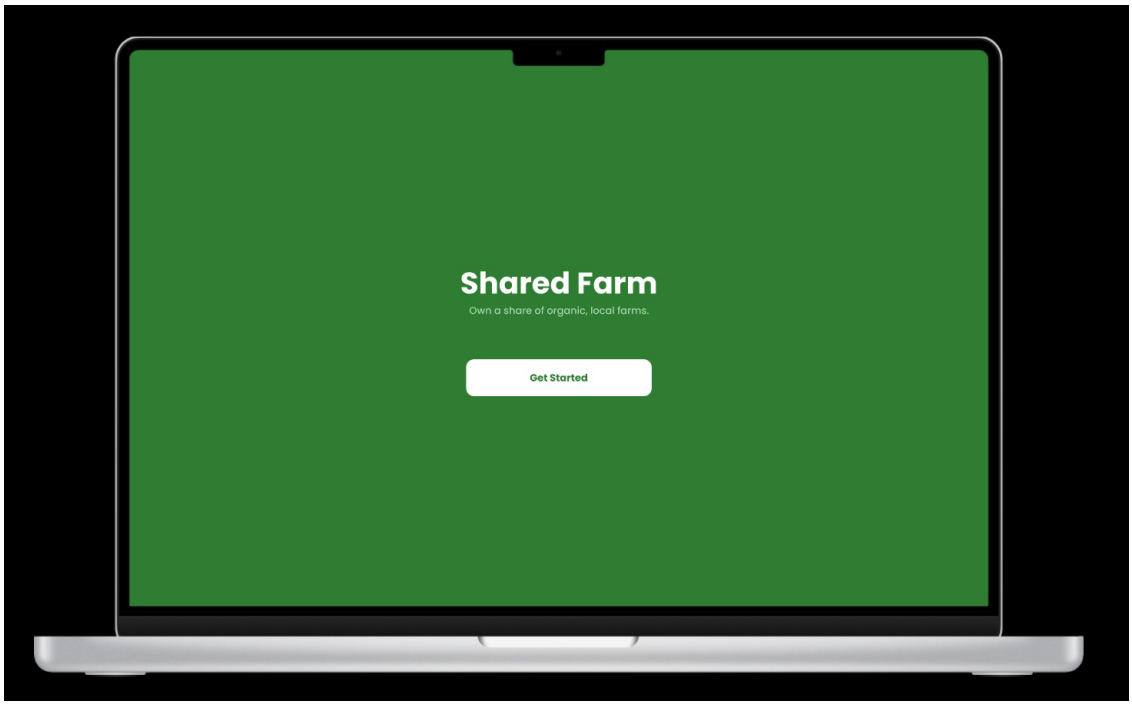
IoT Sensors
(Future): Possible incorporation for live monitoring of farms (livestock well-being, plant development)

### 4.2.3 Human interface

An overview of any human interfaces to be designed for the software is presented.

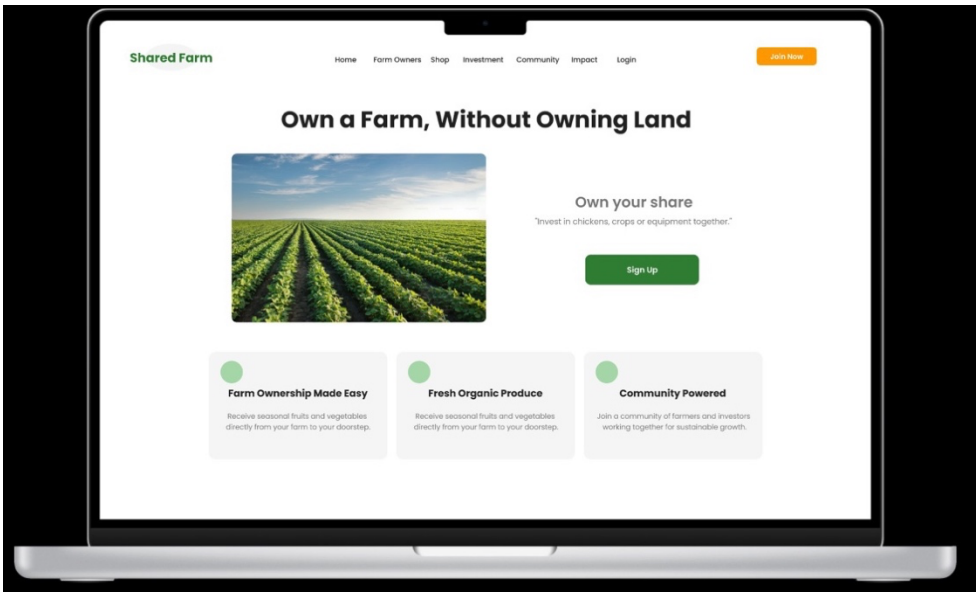
## Splash Screen

This is the entry screen that shows the Shared Farm brand identity with a "Get Started" button to begin using the platform.



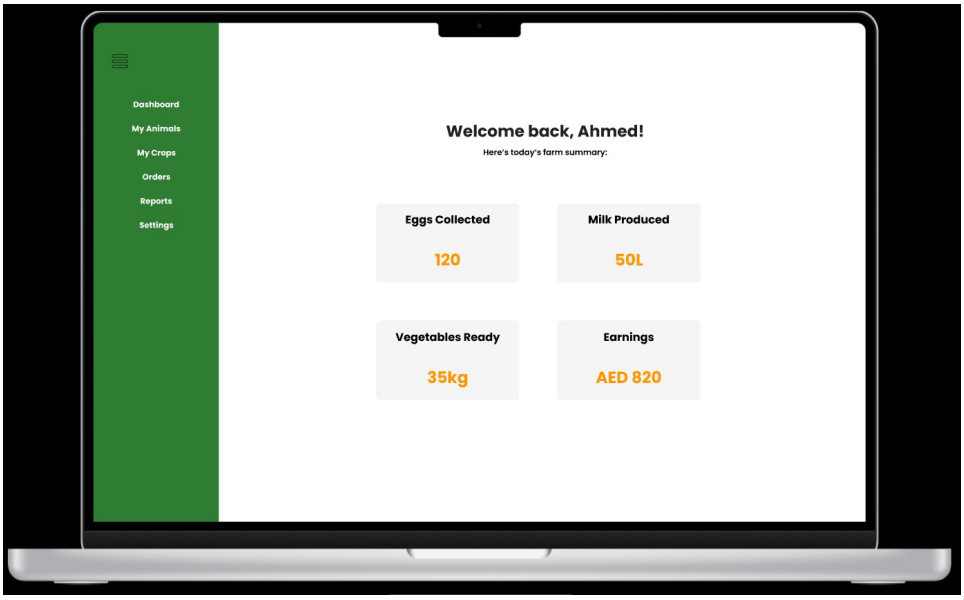

## Home Screen

Introduces the concept of owning a farm without owning land, highlights benefits like farm ownership, fresh organic produce, and community support, with a sign-up option.



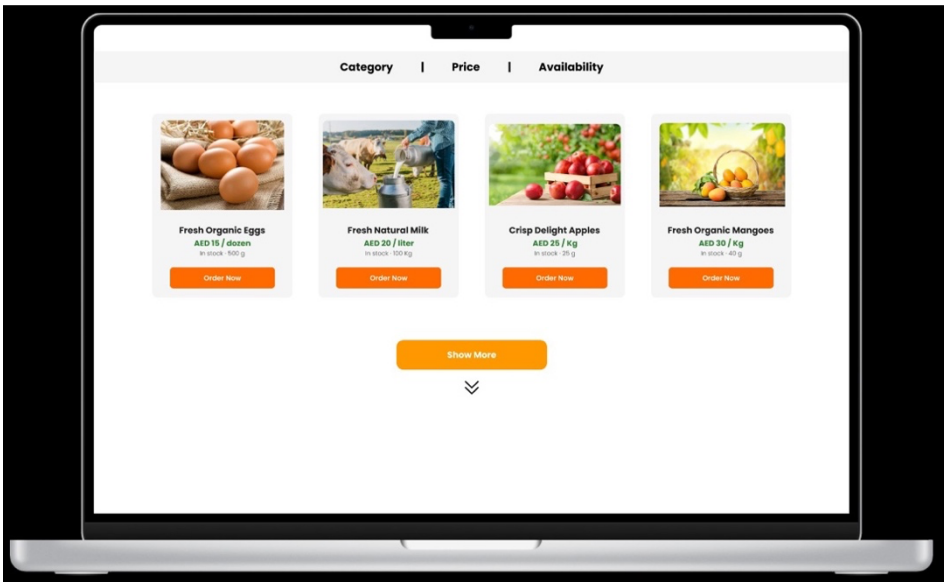
## Dashboard

Displays the user's personalized farm summary, including collected eggs, milk production, vegetables ready, and earnings in a clear overview.



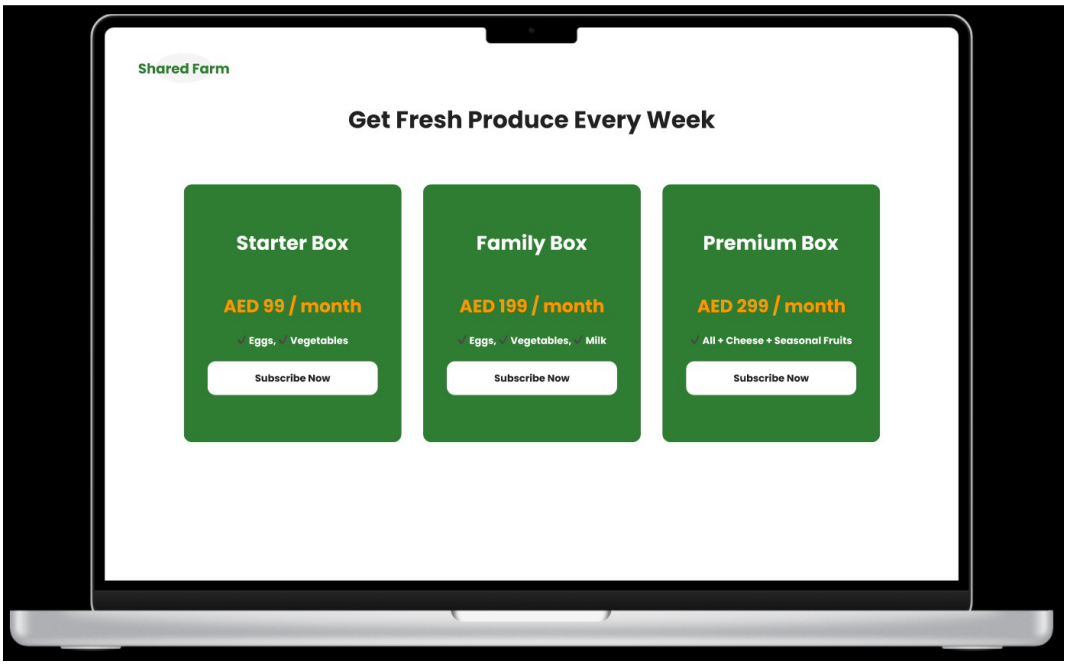

## Product Catalog

Showcases fresh farm products such as eggs, milk, apples, and mangoes with prices, availability, and "Order Now" options.



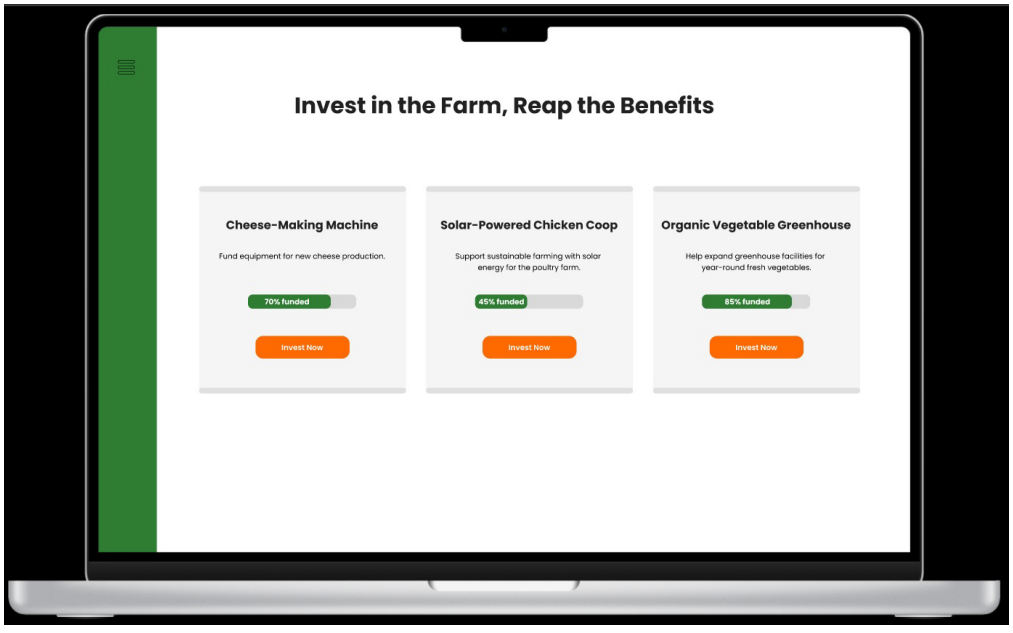
## Subscription Plans

Presents weekly produce box options (Starter, Family, Premium) with fixed monthly pricing and subscription buttons.



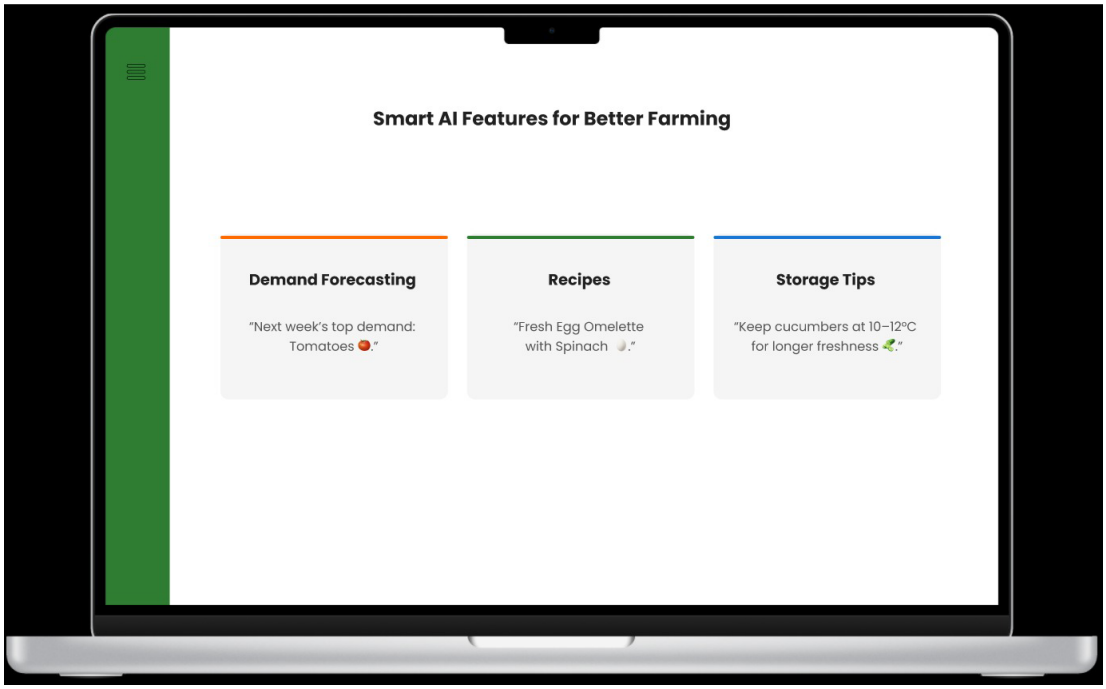

## Investment Opportunities

Allows users to fund specific farm projects like cheese-making machines, chicken coops, or vegetable greenhouses, with progress bars showing funding status.



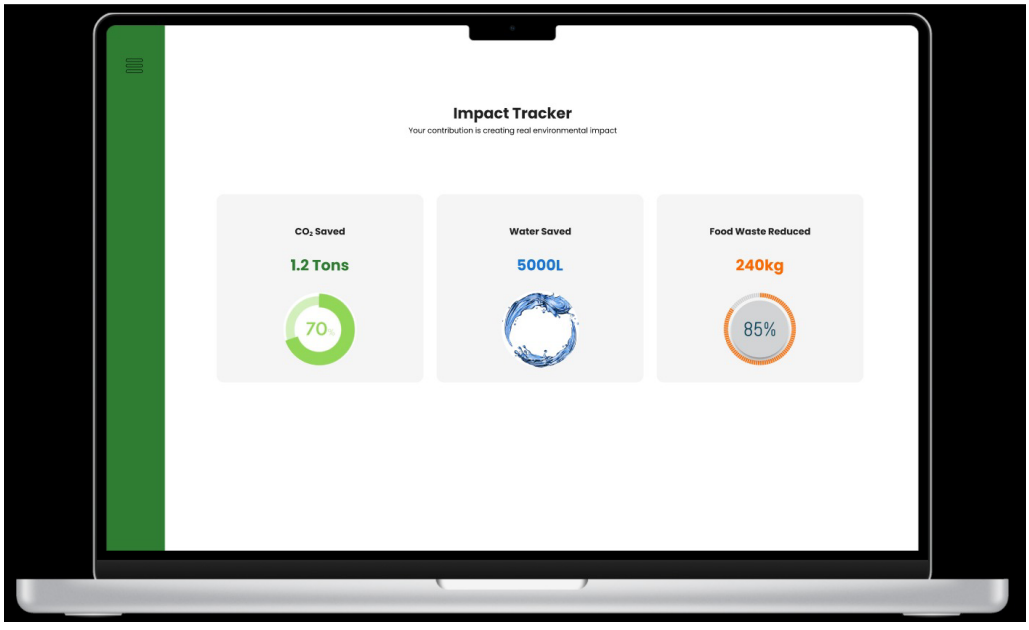
## AI Features

Provides smart tools like demand forecasting, recipe suggestions, and storage tips to support better farming and informed consumer decisions.



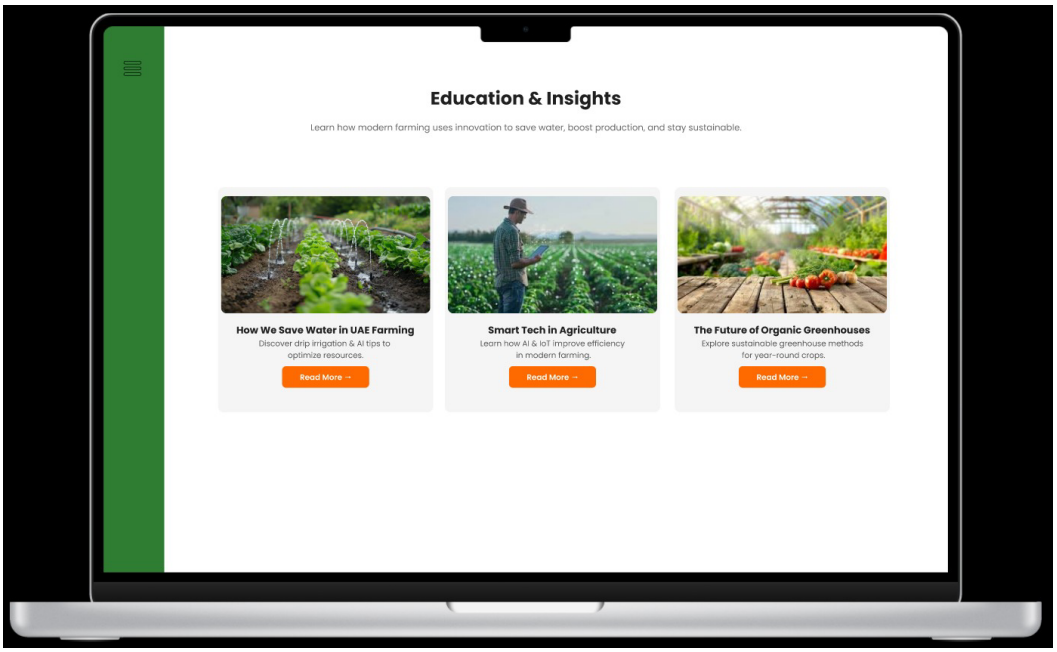

## Impact Tracker

Shows the measurable positive environmental contributions such as $CO_2$ saved, water conserved, and food waste reduced.



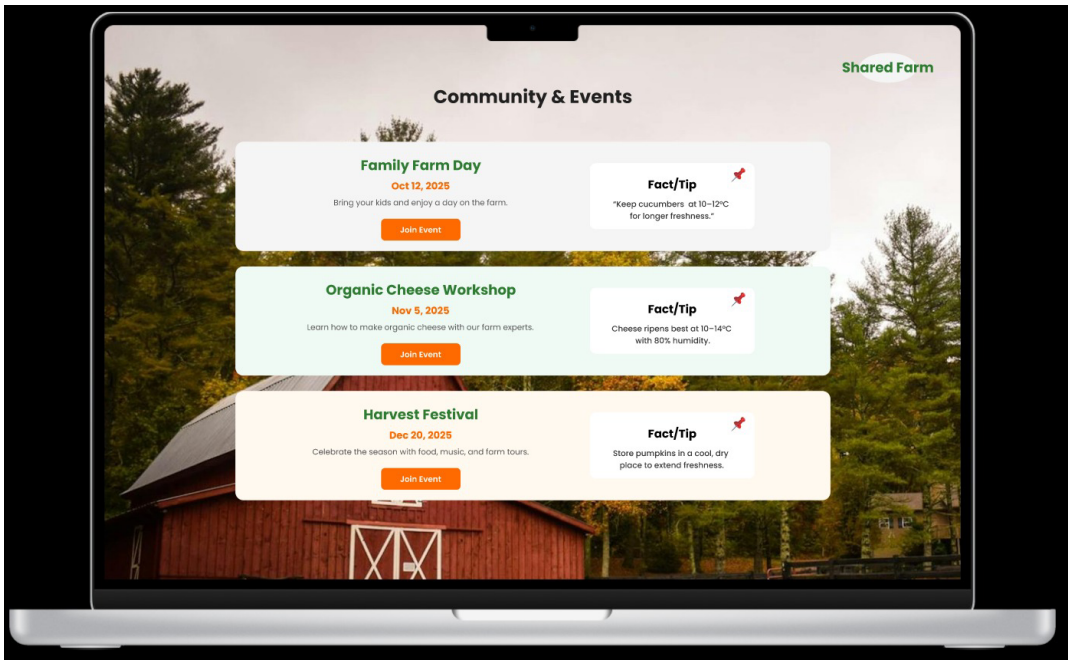
## Education & Insights

Offers blog-style learning resources covering topics like water-saving methods, smart farming technologies, and future greenhouse practices.



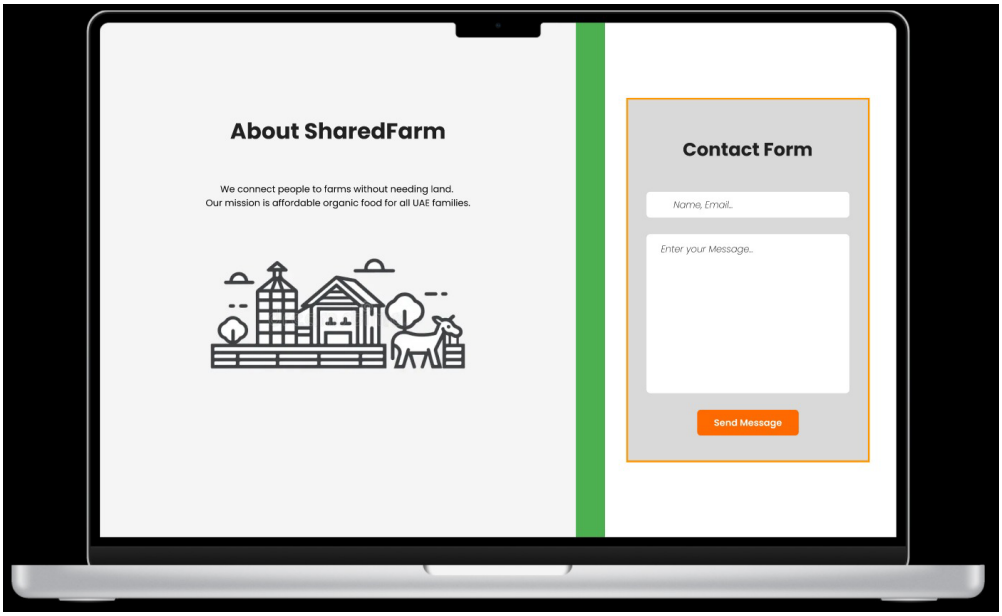

## Community & Events

Lists upcoming farm-related events like Family Farm Day, Cheese Workshops, and Harvest Festivals, with tips and join-event options.



### About & Contact

Explains the Shared Farm mission of connecting people to farms and affordable organic food, with a contact form for inquiries.




# 5.0 Behavioral Model and Description

## 5.1 Description of Software Behavior

### 5.1.1 Events

*Events for *Register Account**

- User opens registration page.

- `buttonPressed[Register]` user submits name, email, password, role.

- `verifyInformation[valid]` system validates credentials and sends email verification.

- `verifyInformation[invalid]` system rejects and prompts retry.

- `accountCreated` new account is initialized.

- `GoToLoginPage` redirects user to login page after success.

*Events for *Login**

- User opens login page.

- `buttonPressed[Login]` user submits credentials.

- `checkCredentials[valid]` system creates a session and redirects to dashboard.

- `checkCredentials[invalid]` system shows error message.

- `GoToForgotPasswordPage` user chooses password recovery option.

*Events for *Forgot Password**
- User clicks “Forgot Password” link.

- `enterEmail` user submits registered email.

- `sendVerificationCode` system sends code.

- `verifyCode[valid]` system allows password reset.

- `verifyCode[invalid]` system shows error and retries.

- `updatePassword` system saves new password.

*Events for *Browse Catalog & Place Order**
- Customer opens product catalog.

- `selectProduct` view product details.

- `buttonPressed[AddToCart]` product added to cart.

- `checkout[valid]` system processes payment and confirms.

- `checkout[invalid]` system cancels and shows error.

- `trackOrder` customer requests order status.

- `cancelOrder` customer requests cancellation.

*Events for *Claim Produce Entitlement**
- Shareholder opens dashboard.

- `buttonPressed[ClaimProduce]` entitlement request submitted.

- `checkBalance[valid]` system schedules free delivery.

- `checkBalance[invalid]` system rejects request.

- `produceAssigned` items reserved for delivery.

*Events for *List Surplus Produce & Exchange**
- Shareholder clicks “List Surplus.”

- `enterProductInfo` details submitted.

- `validation[valid]` system posts listing.

- `validation[invalid]` system rejects listing.

- `approveOffer` shareholder/admin finalizes exchange.

*Events for *Fund Project**
- Investor opens project page.

- `enterInvestment` pledge submitted.

- `paymentProcessed[valid]` system records investment.

- `paymentProcessed[invalid]` error shown.

- `trackProgress` investor views funding updates.

*Events for *Redeem Investment Benefits**
- Investor opens “My Investments.”

- `buttonPressed[Redeem]` claim submitted.

- `eligibility[valid]` system issues reward.

- `eligibility[invalid]` system denies with error.

*Events for *Voting on Decisions**
- Shareholder opens governance dashboard.

- `openProposal` displays voting options.

- `castVote` shareholder selects option.

- `voteClosed` system tallies results.

- `viewResults` outcome displayed.

*Events for *Delivery Workflow**
- Delivery partner accepts job.

- `statusUpdate[Pickup]` item collected.

- `statusUpdate[Transit]` in progress.

- `statusUpdate[Delivered]` order completed.

- `reportIssue` issue flagged.

### 5.1.2 States

*States for *Register Account**

| State | Description |
|---|---|
| Displaying registration page | User is on registration form. |
| Entering details | User inputs name, email, password, and role. |
| Verifying credentials | System checks duplicates and validity. |
| Verification valid | System accepts info and creates account. |
| Verification invalid | Error message displayed; user retries. |
| Account created | User account initialized in system. |

| | |
|---|---|
| Redirect to login | User forwarded to login page. |

*States for *Login**

| State | Description |
|---|---|
| Displaying login page | User is on login form. |
| Submitting credentials | User enters email and password. |
| Verification valid | Credentials accepted, dashboard shown. |
| Verification invalid | Error message displayed. |
| Redirect to Forgot Password | User goes to password recovery page. |

*States for *Forgot Password**

| State | Description |
|---|---|
| Displaying forgot password page | User has requested password reset. |
| Sending verification code | System sends code to email. |
| Checking verification code | User enters code, system verifies. |
| Code valid | System allows password reset. |
| Code invalid | Error shown, retry option. |
| Updating password | User sets new password. |

*States for *Browse Catalog & Place Order**

| State | Description |
|---|---|
| Viewing catalog | Customer browsing items. |
| Viewing product details | Customer opens product page. |
| Cart pending | Items placed in cart. |
| Checkout processing | Payment being processed. |
| Payment valid | Order confirmed. |
| Payment invalid | Transaction failed. |
| Order processing | System preparing order. |
| Order in transit | Delivery ongoing. |
| Order completed | Order successfully delivered. |

| | |
|---|---|
| Order cancelled | Customer/admin cancelled order. |

*States for *Claim Produce Entitlement**

| State | Description |
|---|---|
| Dashboard opened | Shareholder logged in. |
| Request submitted | Claim submitted to system. |
| Balance valid | System approves request. |
| Balance invalid | System rejects claim. |
| Produce assigned | Entitlement scheduled for delivery. |

*States for *List Surplus Produce & Exchange**

| State | Description |
|---|---|
| Listing pending | Shareholder entering details. |
| Validation valid | Listing approved and posted. |
| Validation invalid | Listing rejected. |
| Offer received | Another shareholder/customer makes offer. |
| Exchange approved | Trade finalized. |

*States for *Fund Project**

| State | Description |
|---|---|
| Project opened | Investor views details. |
| Investment submitted | Pledge entered. |
| Payment valid | System confirms investment. |
| Payment invalid | Transaction rejected. |
| Funding progress updated | Project status updated. |

*States for *Redeem Investment Benefits**

| State | Description |
|---|---|
| Investment view opened | Investor views their investments. |
| Redeem submitted | Investor requests benefit. |
| Eligibility valid | Reward issued. |

| | |
|---|---|
| Eligibility invalid | Claim denied. |

*States for *Voting on Decisions**

| State | Description |
|---|---|
| Proposal opened | Shareholder views governance proposal. |
| Voting in progress | Voting window is open. |
| Voting closed | System tallies votes. |
| Results displayed | Outcome shown. |

*States for *Delivery Workflow**

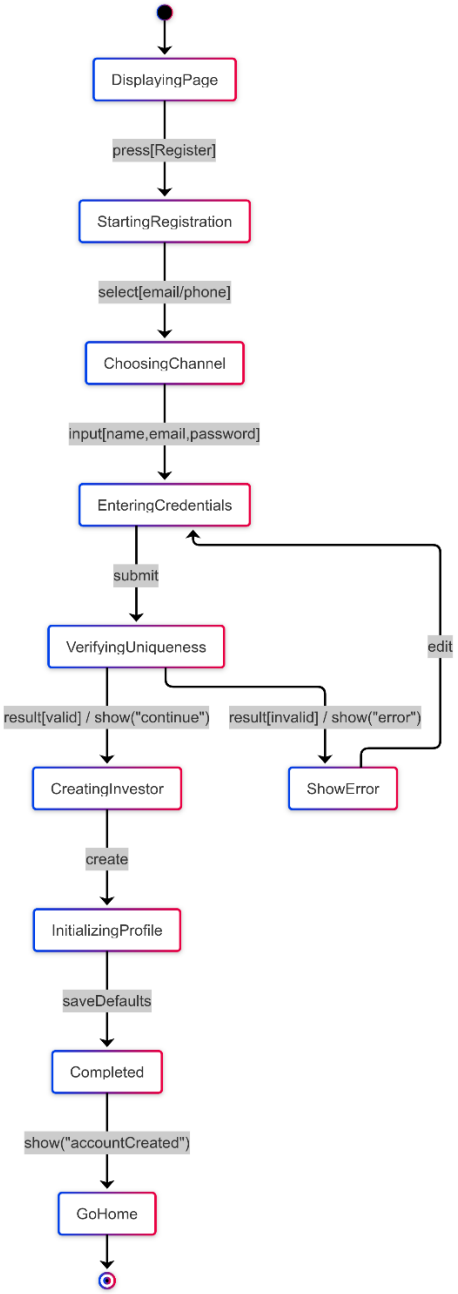
| State | Description |
|---|---|
| Job assigned | Delivery partner accepts order. |
| Pickup complete | Items collected from farm. |
| In transit | Delivery ongoing. |
| Delivered | Order completed successfully. |
| Issue reported | Partner flagged a problem. |

### 5.2 Statechart Diagram

# State diagrams:

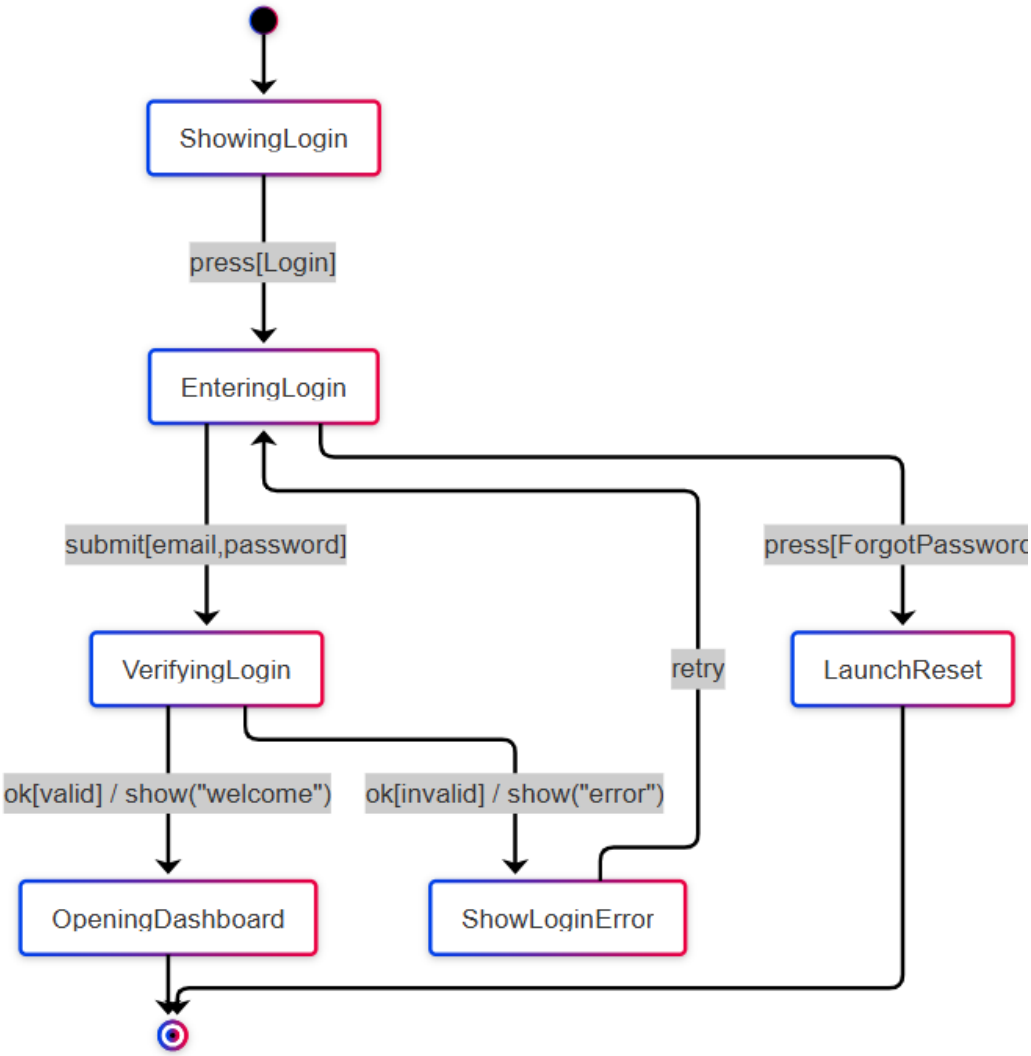
## 1) Register Account:

Covers the process where a user (Customer, Shareholder, Investor, or Admin) registers a new account, chooses a role, and initializes a profile. Invalid or duplicate entries lead to error and retry.



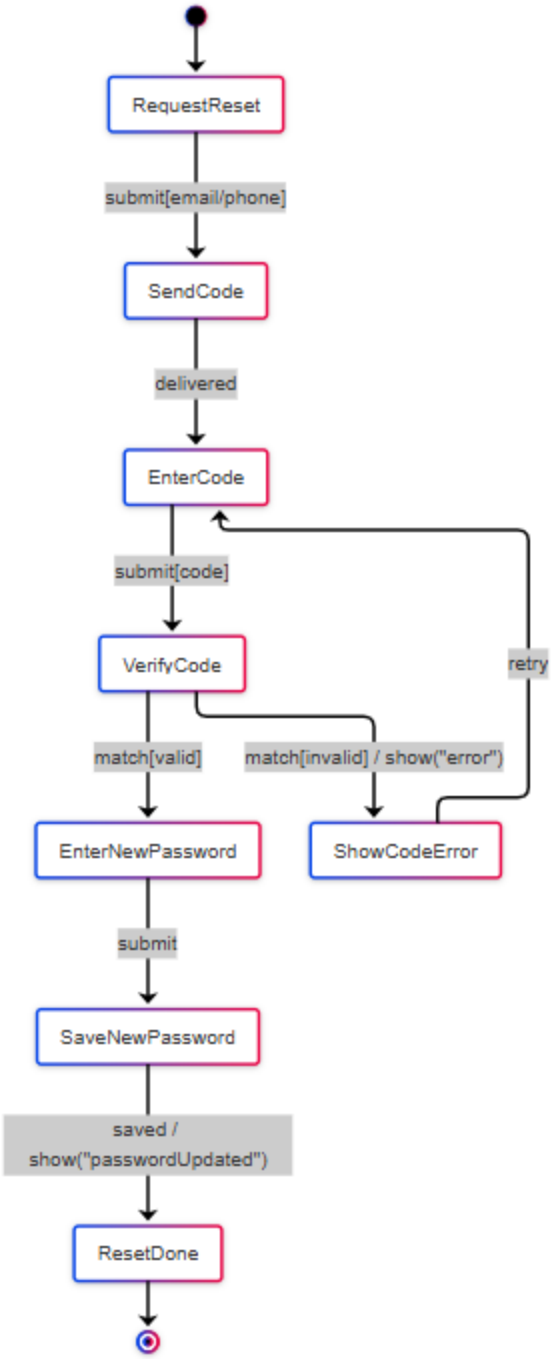
## 2) Login:

Describes how any actor logs into the system by entering credentials. Successful login redirects to their dashboard, while invalid credentials trigger an error or password reset.



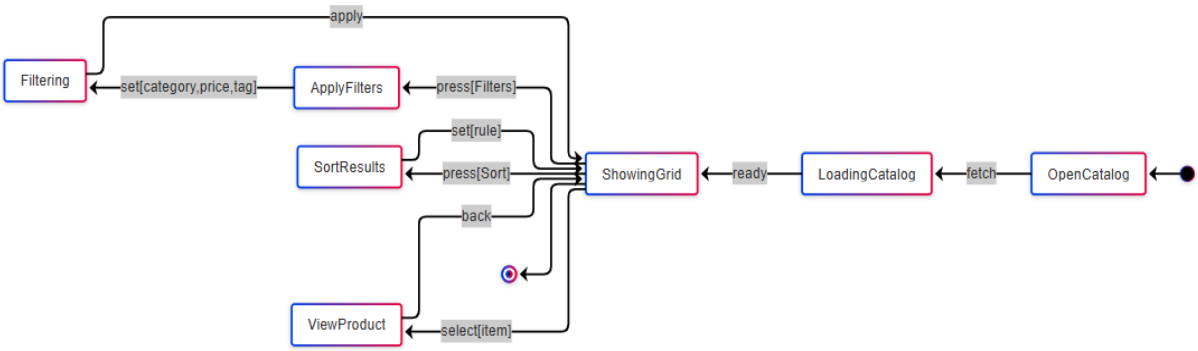

## 3) Forgot Password:

Details how a user resets a password using a verification code. A valid code allows setting a new password; an invalid code prompts retry.



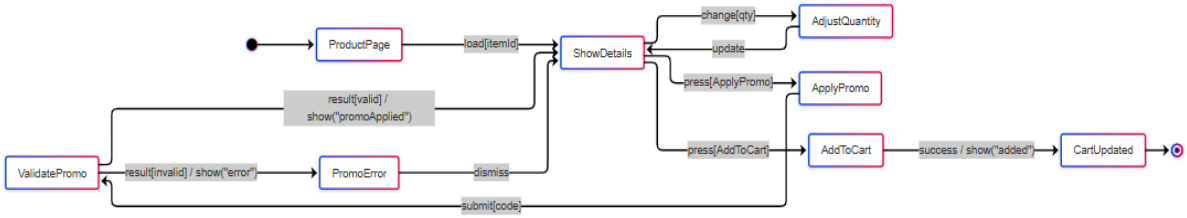

## 4) Browse Catalog & Filter (Customer):

Explains how customers access the catalog, filter or sort products, and open product details. Returning to the grid is always possible.



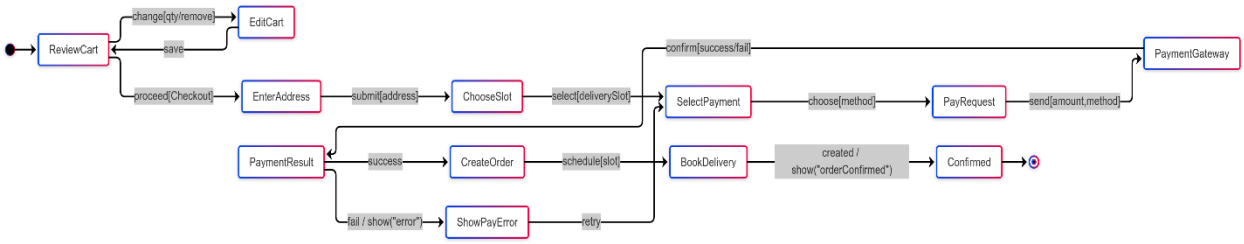

## 5) View Product & Add to Cart (Customer):

Covers actions on a product page such as viewing details, adjusting quantity, applying promo codes, and adding items to the cart.



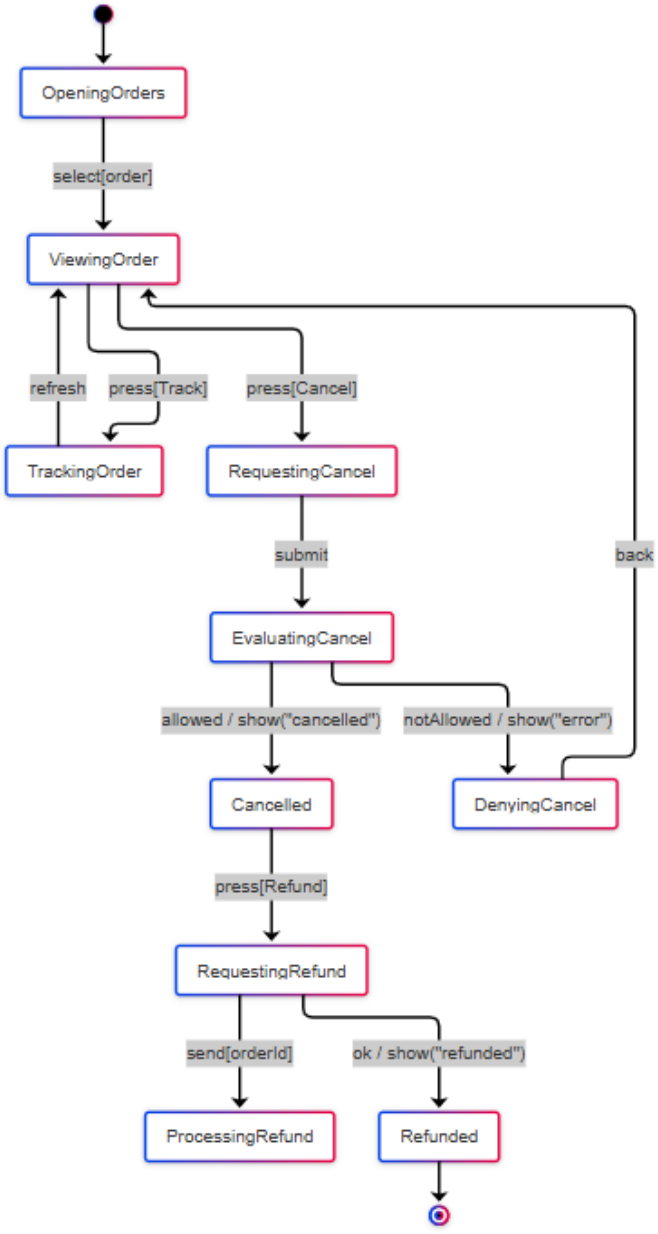

## 6) Checkout Order (Customer):

Describes how customers review their cart, enter address and delivery slot, make payment via the gateway, and receive order confirmation or error.



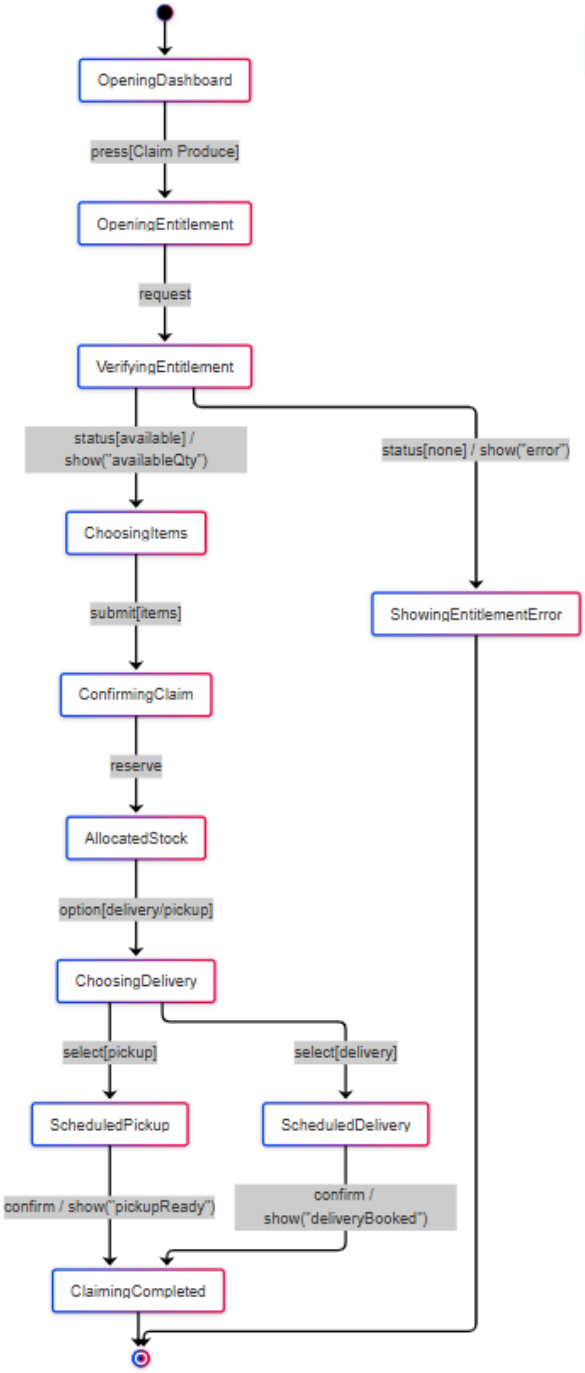
## 7) Track / Cancel / Refund Order (Customer):

Outlines how a customer can track order status, request cancellation, and if eligible, request a refund that is processed by the system.



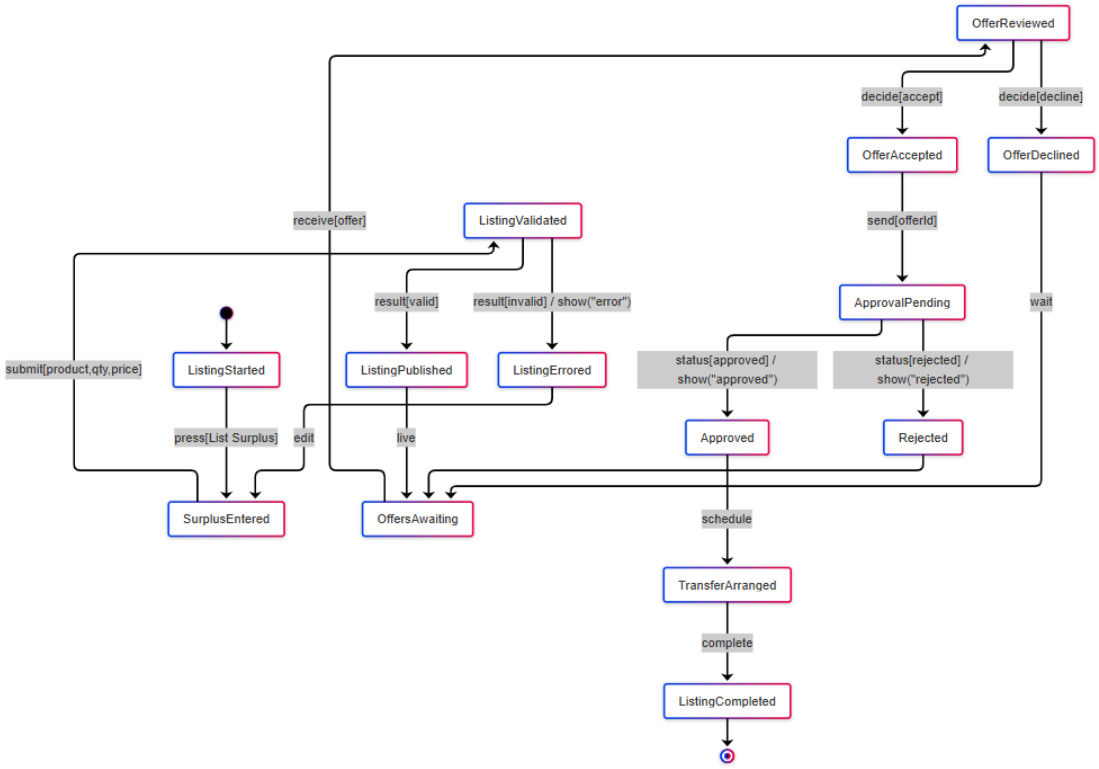
## 8) Claim Produce Entitlement (Shareholder):

Models how shareholders claim their produce entitlements, with the system verifying availability and arranging delivery or pickup.



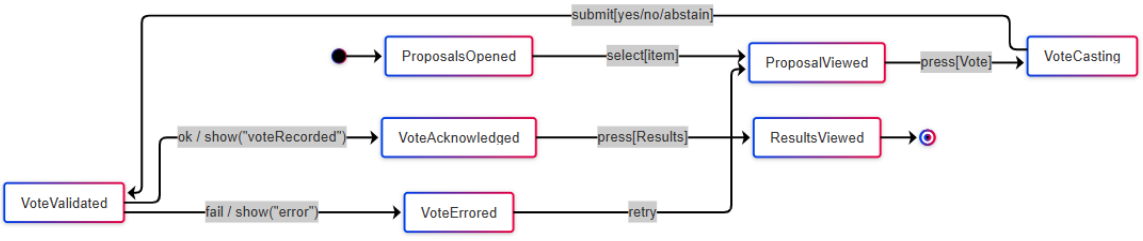
## 9) List Surplus Produce & Approve Exchange Offer (Shareholder + Admin):

Represents how shareholders list surplus products, handle exchange offers, and admins approve or reject these offers before transfer.



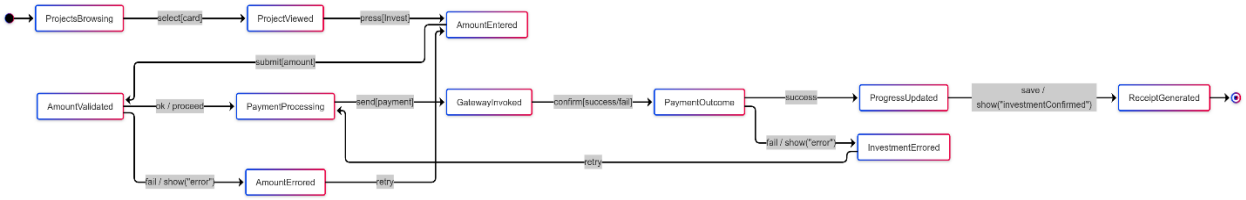

## 10) Vote on Decisions (Shareholder Governance):

Shows how shareholders open proposals, cast votes, and receive acknowledgement. Results can be viewed once tallies are updated.



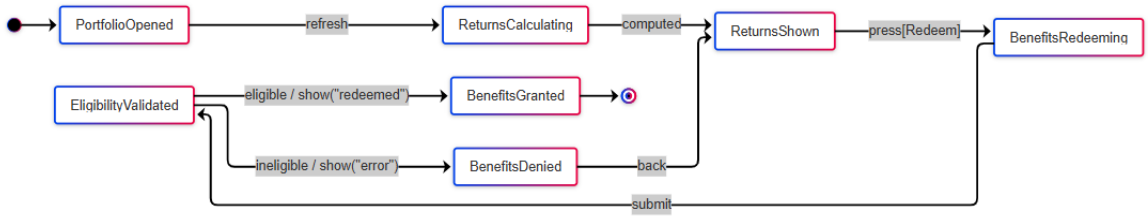
## 11) Invest in Project (Investor):

Covers how investors browse projects, enter investment amounts, validate them, pay via gateway, and receive confirmation or error.



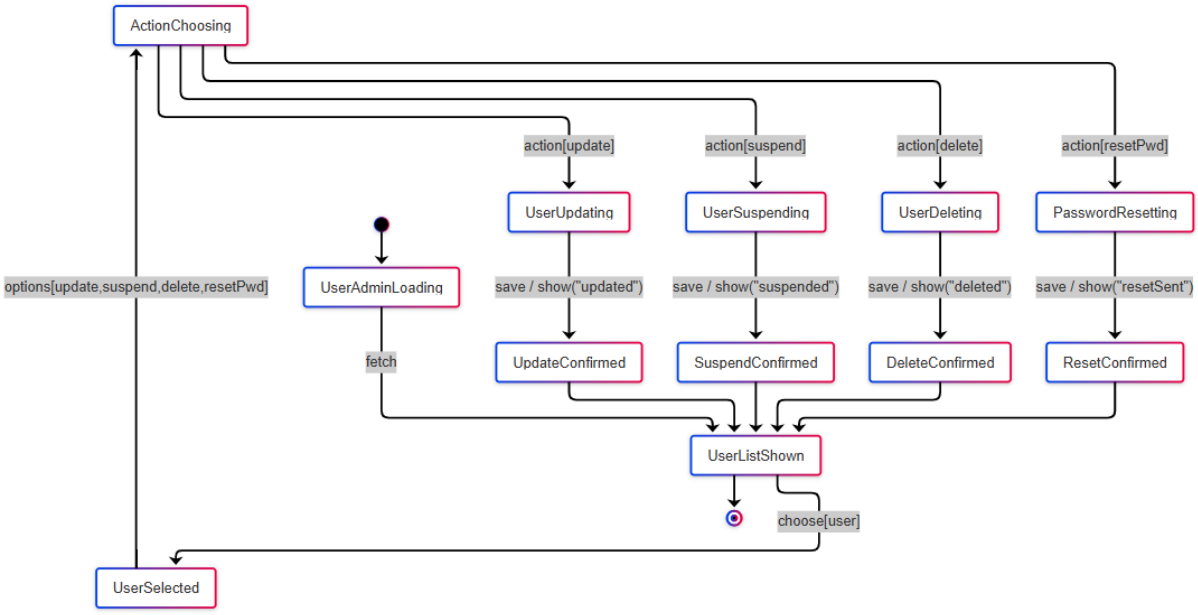

## 12) Track Returns & Redeem Benefits (Investor):

Explains how investors check their returns and redeem benefits if eligible. Ineligible cases trigger an error message.



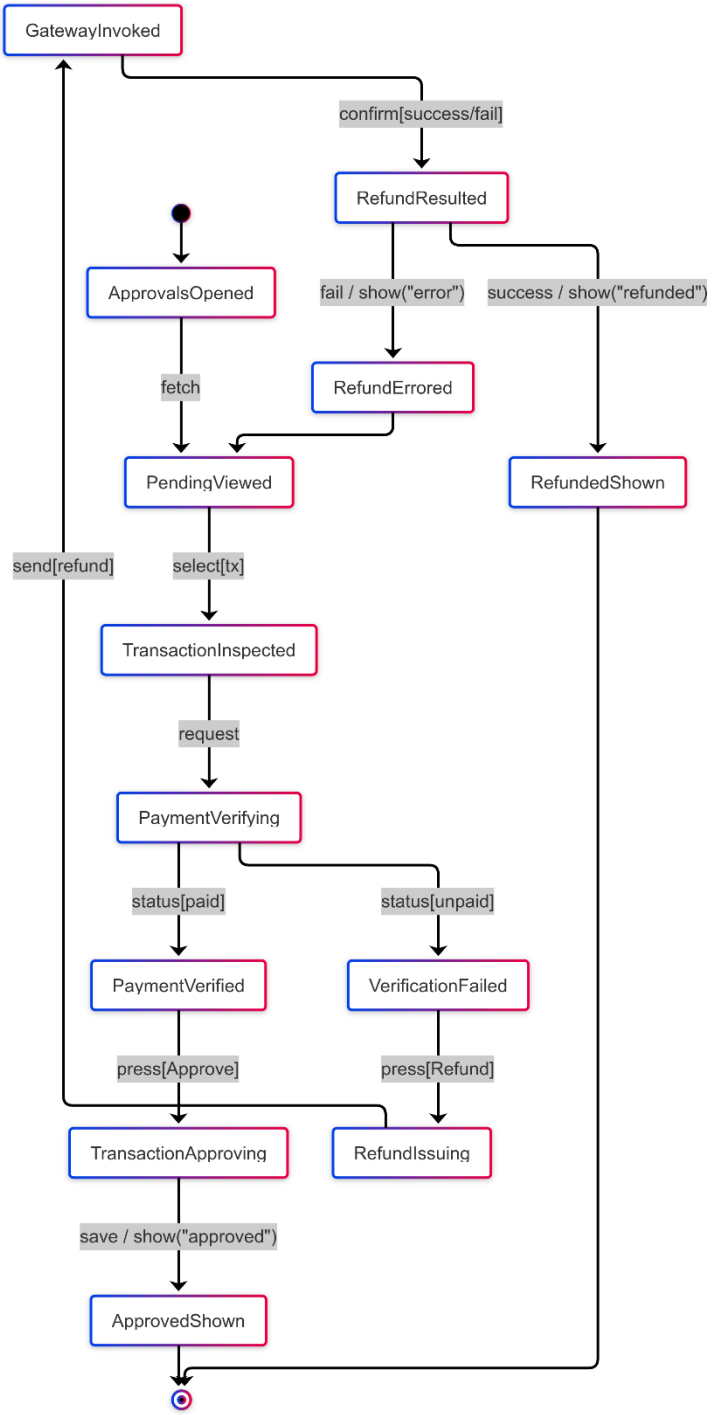
## 13) Manage Users (Administrator):

Models the admin's ability to view user lists, update details, suspend or delete accounts, and reset passwords, with confirmations after each action.



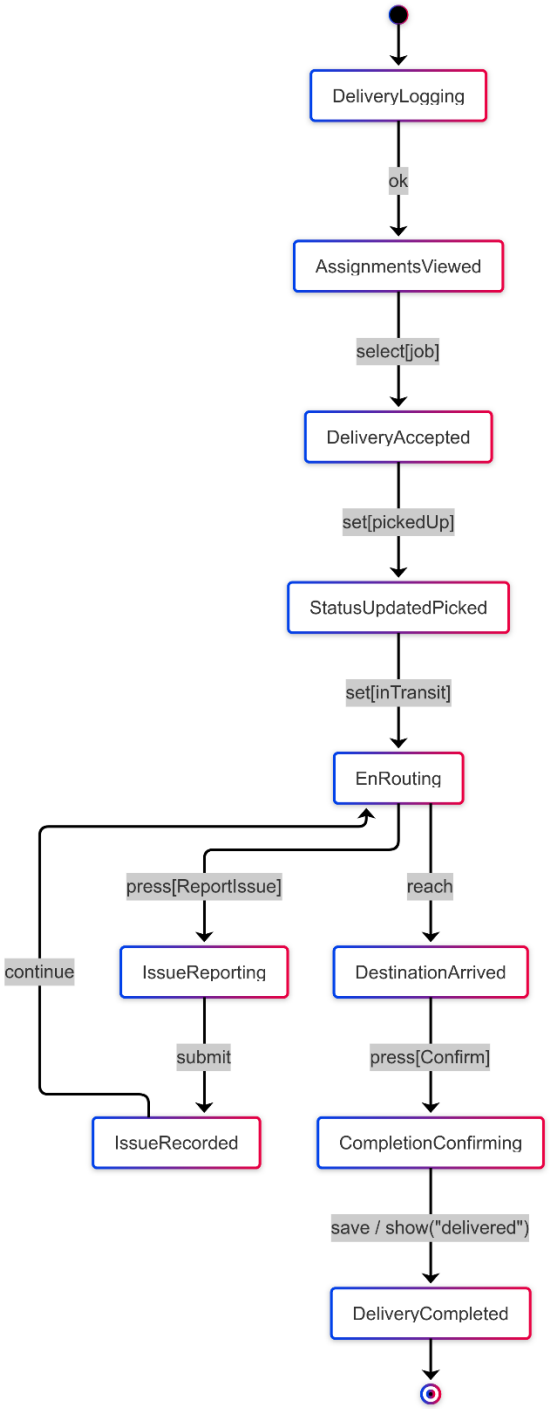
## 14) Approve Transactions & Issue Refund (Administrator + Gateway):

Covers how admins inspect pending transactions, verify payments, approve them, or process refunds through the payment gateway.



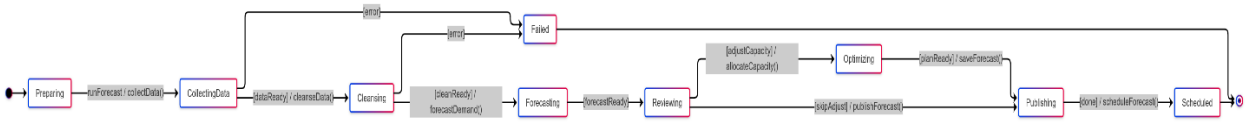
## 15) Delivery Workflow (Delivery Partner):

Details the delivery partner's process of accepting a delivery job, updating statuses from pickup to in-transit, reporting issues if needed, and confirming completion.



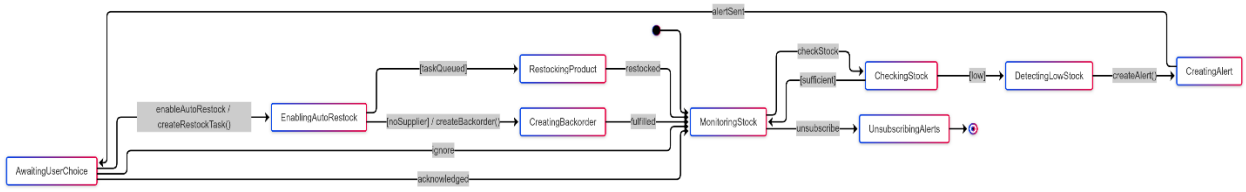
## 16) Demand Forecasting:

The system starts idle, collects and cleans data, forecasts demand, and optimizes capacity before publishing and scheduling the next run. On successful publication it returns to idle with ok; on any error it moves to Failed and then resets.




## 17) Stock & Order Automation:

The system monitors inventory, checks thresholds, and when stock is low it creates alerts and can queue auto-restock tasks. After restock or cancel it returns to monitoring. Normal checks loop with sufficient; low stock triggers alert and optional restocked; invalid paths return error.



## 6.0 Restrictions, Limitations, and Constraints

The main limitations and difficulties that influenced the specification, design, and implementation of the Shared Farm Web Application are listed in this section.

1. **User Perception of Shared Ownership (Specification)**
   The idea of co-owning farm shares via a digital platform might be foreign to many consumers. Because people may link this unfamiliarity to questionable investment schemes, it may cause hesitancy or suspicion. To address this, the system must emphasize transparency through traceable transactions, clear reporting, and reliable order management.

2. **Regulatory Adherence (Specification)**
   The application needs to abide by UAE laws related to investment activities, food safety, and e-commerce. Depending on local financial and agricultural regulations, some aspects, such community-based finance models or investor profit-sharing, may be restricted. These regulatory requirements define boundaries for what can be specified and implemented in the system.

3. **Dependence on External Services (Design)**
   The system relies on third-party services such as payment gateways, cloud storage, and notification platforms. The application's functionality and performance will suffer if any of these external services are unavailable, restricted, or incompatible. Because the platform needs to be adaptable enough to manage possible disruptions, this dependence places restrictions on its design.

4. **Scalability of the Platform (Design)**
   A small number of users and transactions will be used to test the first version of the system. Although this facilitates early validation, it limits the capacity to ensure seamless operation when dealing with higher transaction loads or bigger user bases. Additional architectural design modifications will be needed to accommodate future scalability, however they are not included in the present version.

5. **Accuracy of Artificial Intelligence Features (Implementation)**
   The Web Application incorporates AI for recipe recommendations, demand predictions, and storage advice. However, the quality of input data, the unpredictability of consumer behavior and agricultural productivity, and other factors restrict how accurate these

features can be. Until the AI models are regularly enhanced through practical use, this implementation limitation can lead to less trustworthy recommendations.

# 7.0 Validation Criteria

This section describes how we will test our shared farm ownership platform, the expectations we have for it, and its performance boundaries.

## 7.1 Classes of Tests

We will perform multiple types of testing. The most important ones are:

**1. Unit Testing:** Testing small modules like produce tracking, recipe suggestions, and investment transactions before integrating them.
**2. Functional Testing:** Testing the platform to ensure it meets requirements such as stock automation, dashboards, and subscriptions.
**3. Performance Testing:** Testing how the system handles high numbers of users, transactions, and farm data updates.
**4. Security Testing:** Testing how the platform protects sensitive user data, transactions, and prevents unauthorized access.
**5. Usability Testing:** Testing the user interface (UI) to ensure it is intuitive for both farm owners and customers.

## 7.2 Expected Software Response

**1. Unit Testing:**

-  **Expectations:** Each sub-system should work correctly before integration.
-  **Example:** The AI demand forecasting module should generate accurate predictions before connecting with the stock automation system.

**2. Functional Testing:**

-  **Expectations:** The system should meet all requirements defined during the requirements phase.
-  **Example:** Owners should be able to view their dashboard, track their share of crops/animals, and receive correct reports. Customers should be able to subscribe to produce boxes or order online easily.

**3. Performance Testing:**

-  **Expectations:** The platform should handle large numbers of orders, farm updates, and transactions simultaneously without crashing.

 **Example:** Hundreds of customers should be able to place orders during peak times without delays.

**4. Security Testing:**
 **Expectations:** The system should prevent fraud, data leaks, and unauthorized access to farm investment or user dashboards.
 **Example:** Before processing a payment or farm investment, the system validates user identity and transaction details.

**5. Usability Testing:**
 **Expectations:** The UI should be simple and easy to use for all types of users, including farmers, investors, and customers.
 **Example:** A new farm member can quickly understand how to track their crops and place free orders without confusion.

## 7.3 Performance Bounds

 **Response Time:** The system will respond to user queries (Example: farm stock updates, order status, dashboards) in real-time or within a few seconds. This ensures smooth experience during peak order times.

 **Workload:** The platform can handle thousands of transactions, stock updates, and log entries at once. For example, when many customers place orders after harvest announcements.

 **Latency:** Farm updates (such as crop growth, product availability, or equipment investment) should reflect in dashboards within seconds. AI suggestions like recipes or storage tips will also update instantly.

 **Scalability:** The platform is cloud-based, allowing automatic scaling to support more farms, members, and customers as demand grows in the UAE.

 **Fault Tolerance:** The system can recover quickly from server or network issues without losing data. Backup and replication ensure that farm ownership data, investments, and transaction history are always preserved.

**Section 8**

**Leader of phase 1:**

**Evaluator:**

## 8.0 Credits and Work Distribution.

This section acknowledges the contributions of the team members who participated in the system requirements phase.

## 8.1 Credits for Diagram, UI, presentation.

| **Diagram** | **Contributor** |
|---|---|
| *Use case Diagram* | All team members |
| *Sequence Diagram* | All team members |
| *Activity Diagram* | Hind, Amna |
| *Data Model Diagram* | Hasna |
| *Class Diagram* | All team members |
| *State chart Diagram* | Reyam |
| *User Interface* | Asma, Namah |
| *Presentation Design* | Alia |

## 8.2 Credits for Report Writing.

| **Section Number** | **Contributor** |
|---|---|
| *Section 1* | Asma, Namah |
| *Section 2* | All team members |
| *Section 3* | Alia, Reyam |
| *Section 4* | All team members |
| *Section 5* | Hasna, Hind |
| *Section 6* | Hind Almarzooqi |
| *Section 7* | Amna Almazrouei |